



Sistema de Monitoreo de Acciones Peligrosas en el Conductor - Aprendizaje No Supervisado

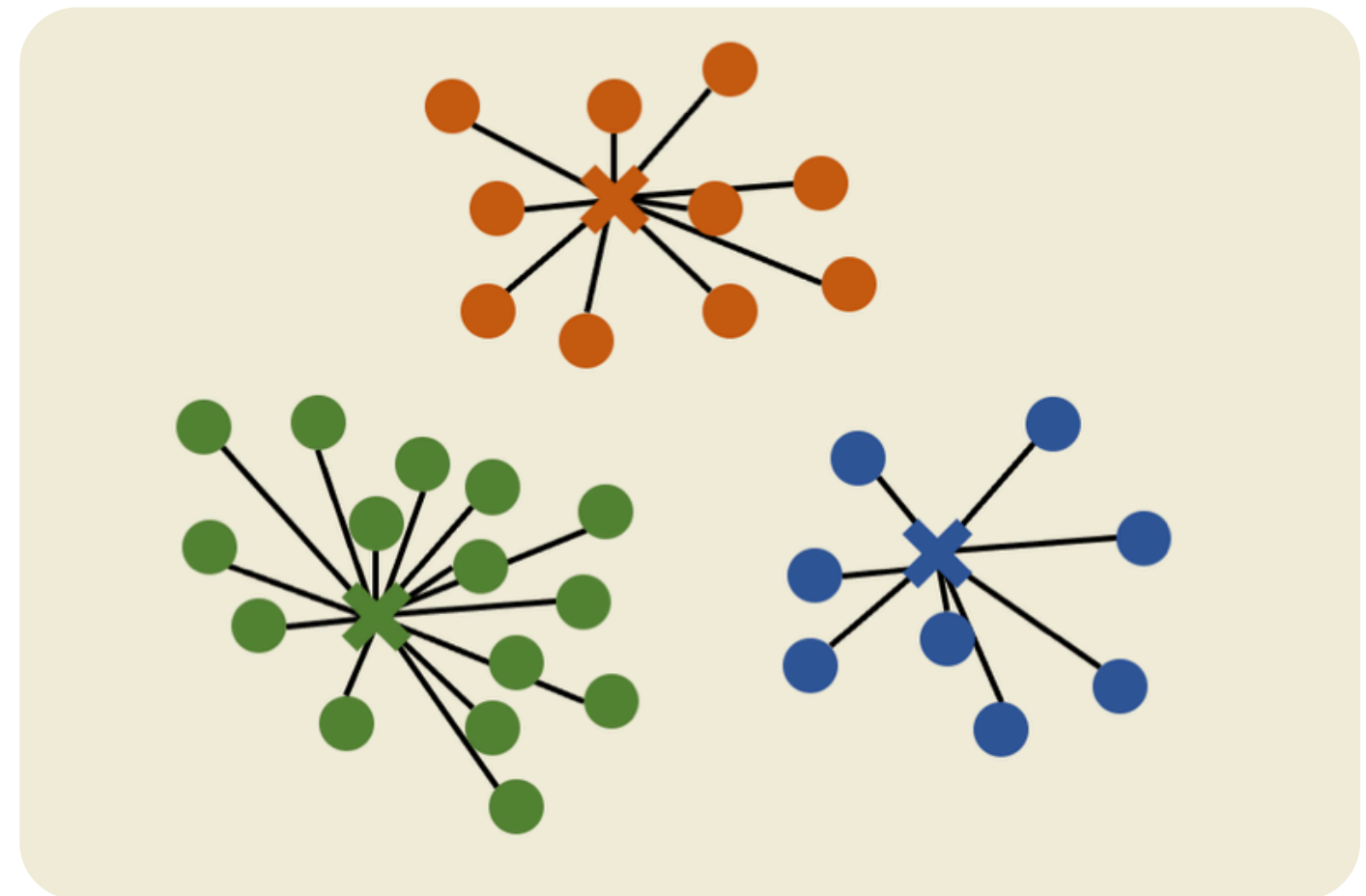
Iván Augusto Camargo López - 2230033

Santiago Torres - 2202024



Índice

1. ¿En dónde vamos?
2. Reducción de Dimensiones: PCA/TSNE
3. Reducción de Dimensiones: Otros Algoritmos
4. Métodos de Agrupamiento: K-Means, DBSCAN y Agglomerative Clustering
5. Métodos de Agrupamiento: Otros Algoritmos



1. ¿En dónde vamos?



- Recordando, para este momento se han extraído las posturas de 16 sujetos distintos, Procedentes de la utilización de VitPose y MediaPipe para la identificación de landmarks
- Ya tenemos el dataset. Es momento de cargarlo para realizar un análisis desde el punto de vista del aprendizaje no supervisado.

Top Mejores Resultados

(anteriores)



RandomForestClassifier

```
max_depth=200,  
min_samples_split=2,  
min_samples_leaf=1,  
criterion='gini',  
class_weight='balanced'
```

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Accuracy	<u>0.863</u>	<u>0.810</u>	<u>0.970</u>
train_Precision	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Precision	<u>0.863</u>	<u>0.803</u>	<u>0.970</u>
train_Recall	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Recall	<u>0.863</u>	<u>0.810</u>	<u>0.970</u>
train_F1_Score	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_F1_Score	<u>0.859</u>	<u>0.789</u>	<u>0.968</u>
train_Cohen_Kappa	<u>0.964</u>	<u>0.969</u>	<u>0.985</u>
test_Cohen_Kappa	<u>0.682</u>	<u>0.591</u>	<u>0.847</u>

1. ¿En dónde vamos?

- Sin embargo, en este caso, había un problema: **no estábamos particionando correctamente el train/test por sujetos**
- Esto llevaba a que, por ejemplo, datos del mismo sujeto estén en train y test. En otras palabras, **data leakage**.
- Hay que hacer que el modelo recuerde cómo identificar **situaciones peligrosas**, **no recordar el sujeto**
- Por eso, las nuevas particiones son de 13 sujetos para train, y 3 para test



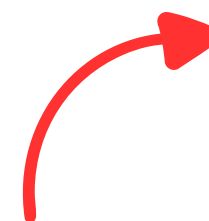
Top Mejores Resultados

(ahora sí)



RandomForestClassifier

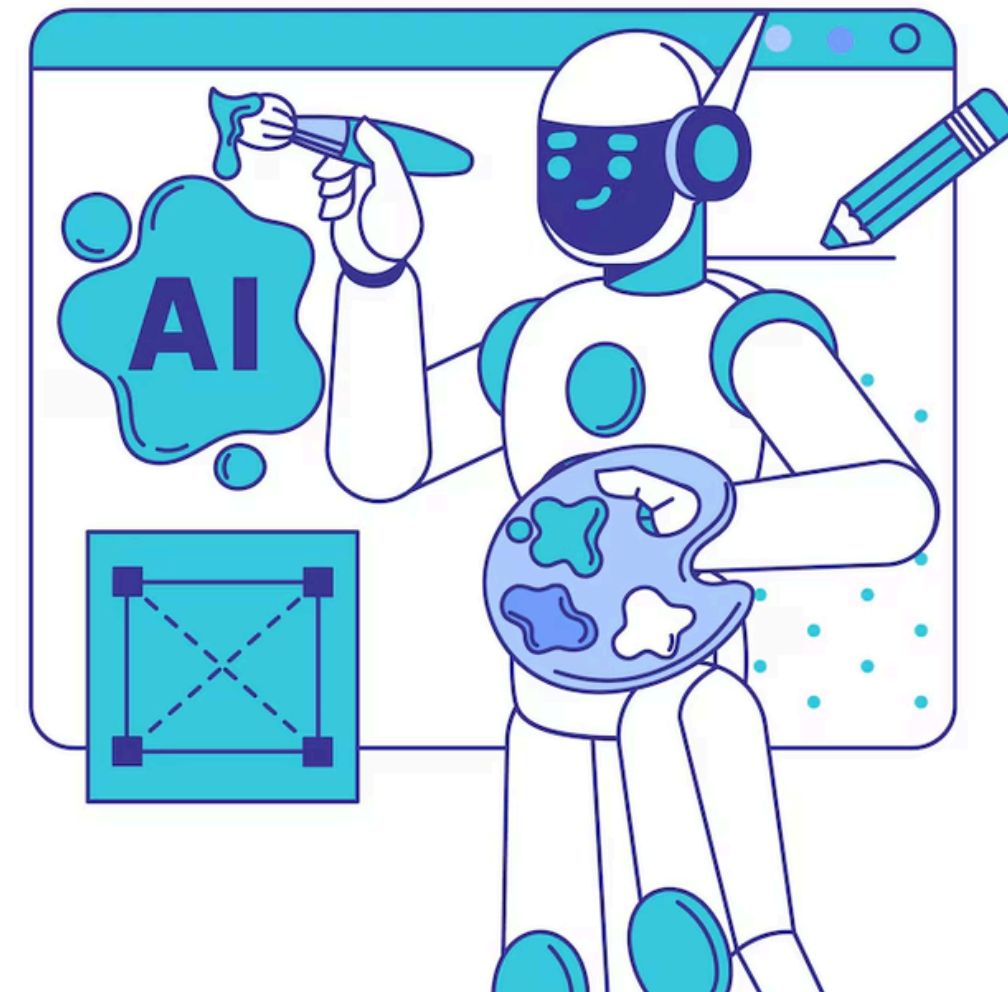
```
max_depth=200,  
min_samples_split=2,  
min_samples_leaf=1,  
criterion='gini',  
class_weight='balanced'
```



	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.616</u>	<u>0.683</u>	<u>0.860</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.594</u>	<u>0.550</u>	<u>0.806</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.616</u>	<u>0.683</u>	<u>0.860</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.604</u>	<u>0.597</u>	<u>0.824</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>-0.015</u>	<u>-0.005</u>	<u>0.004</u>

2. Reducción de Dimensiones: PCA/TSNE

- Se utilizaron estas técnicas para **reducir dimensionalidad** de nuestro dataset.
- En nuestro caso, estas herramientas son **importantísimas**, porque tenemos muchas features, **que suponen mucho costo computacional**, y no todas ellas son útiles para la clasificación; como se verá, hay muchas que generan ruido



PCA - 1/4 de las características originales

- En este caso, no solamente se disminuyeron las features; **incluso aumentó el puntaje en las métricas**
- En otras palabras, se eliminó el ruido que algunas de las dimensiones producían.
- Entonces, la pregunta es: ¿hasta cuándo esta reducción nos resultará beneficiosa?



	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.707</u>	<u>0.721</u>	<u>0.886</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.605</u>	<u>0.579</u>	<u>0.785</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.707</u>	<u>0.721</u>	<u>0.886</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.618</u>	<u>0.605</u>	<u>0.832</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>0.004</u>	<u>0.000</u>	<u>0.000</u>

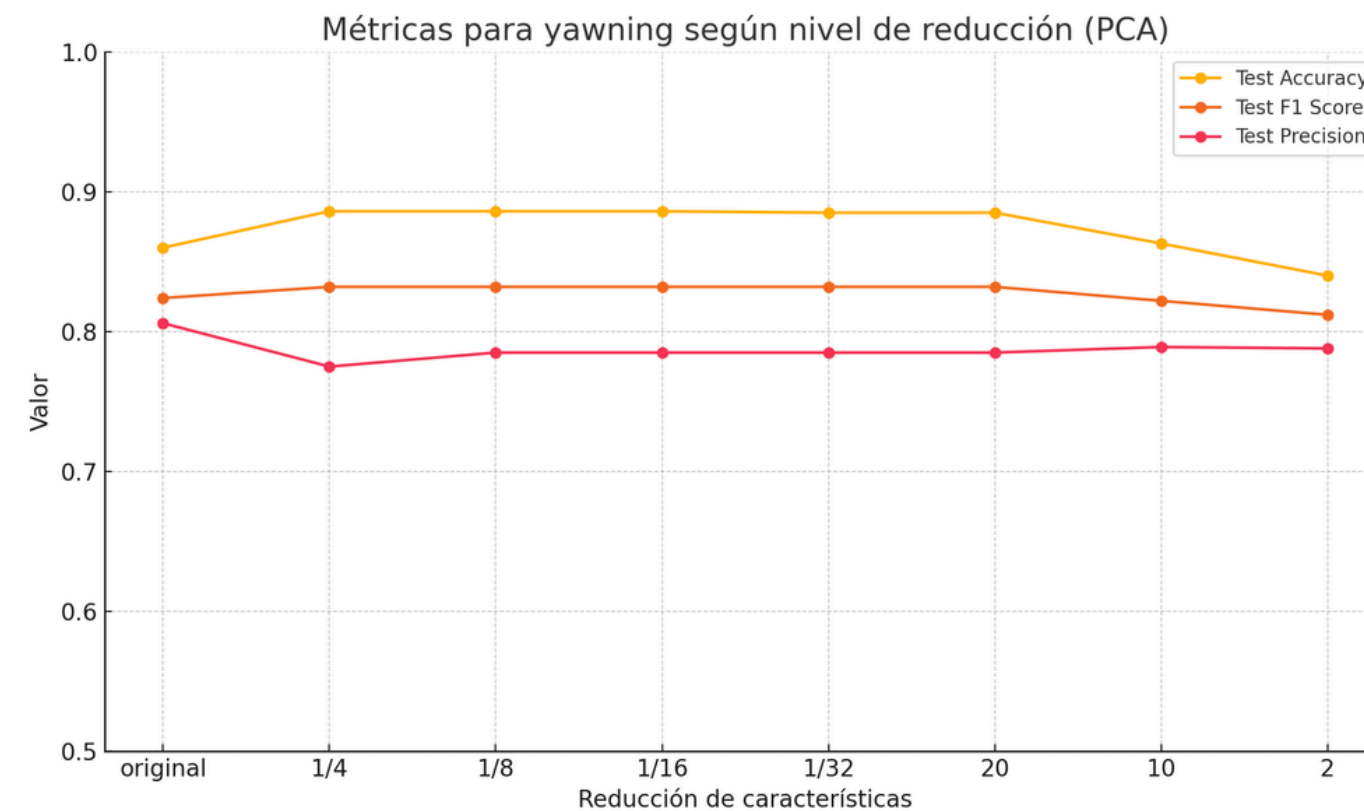
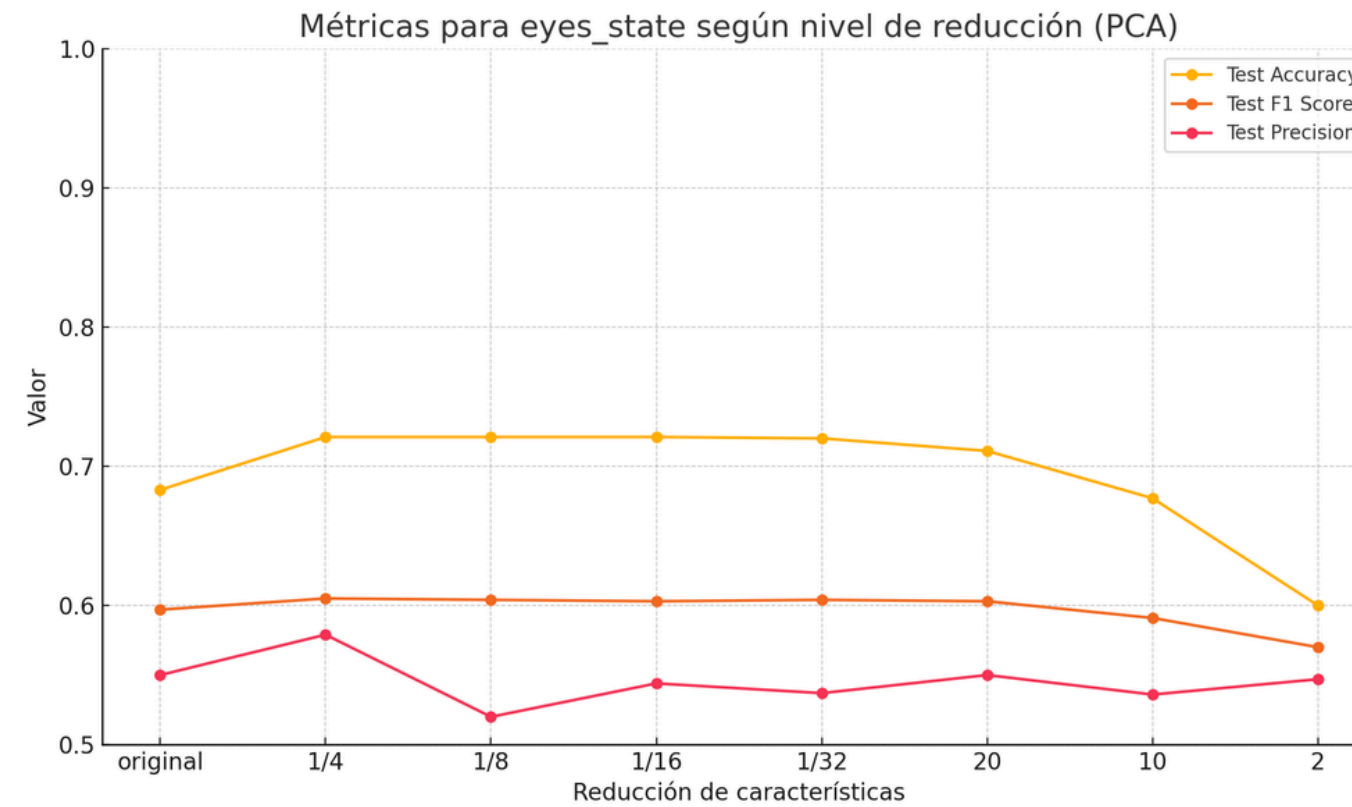
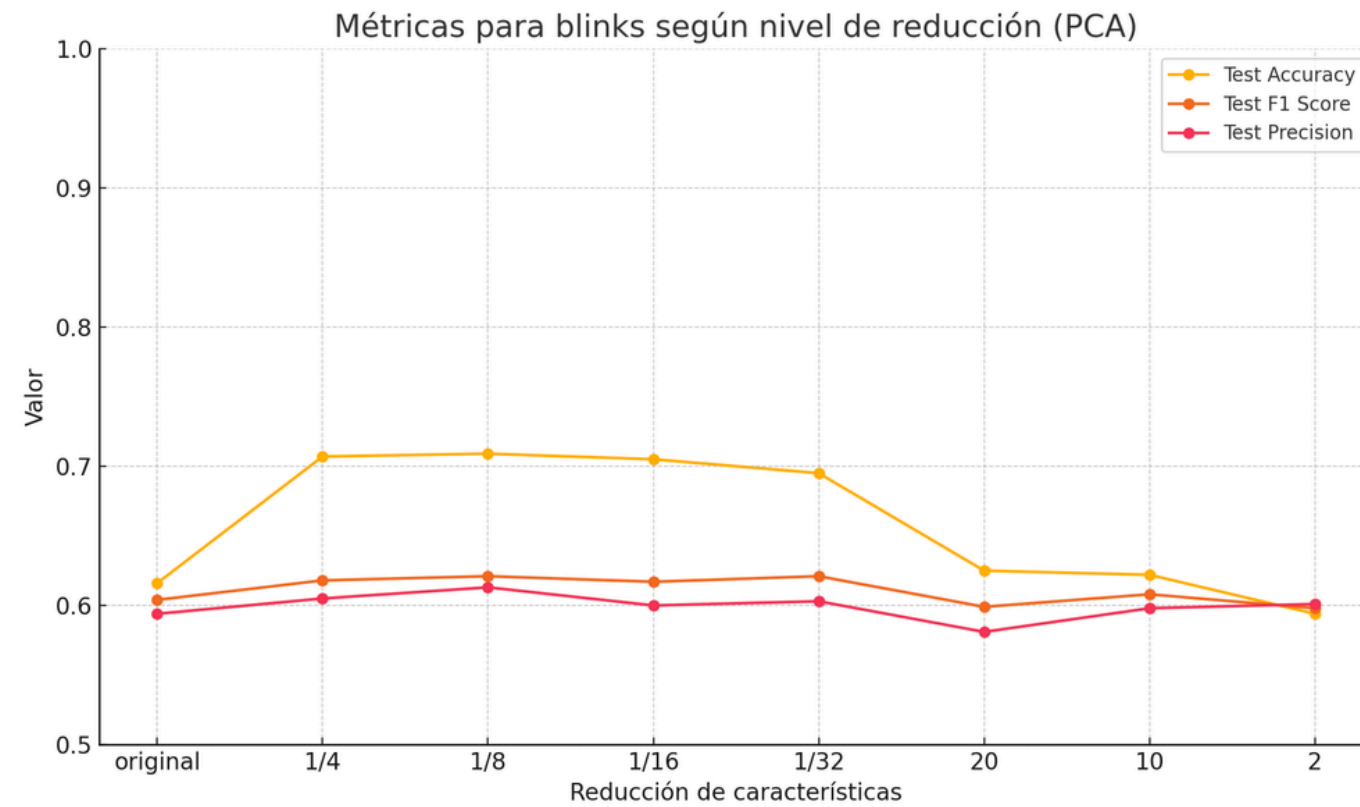
PCA - Exactamente 2 componentes

- Este es el caso del otro extremo: reducirlas a únicamente **2 componentes**
- Para este punto, las métricas bajan con respecto a la situación inicial.
- Sin embargo, la disminución no es tan drástica como se esperaría disminuir de **1450 features a únicamente 2**
- Esto prueba la eficiencia del PCA en problemas como el nuestro

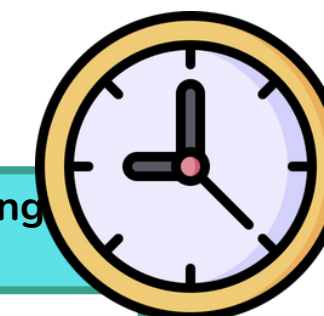


	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.594</u>	<u>0.600</u>	<u>0.840</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.601</u>	<u>0.547</u>	<u>0.788</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.594</u>	<u>0.600</u>	<u>0.840</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.598</u>	<u>0.570</u>	<u>0.812</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>0.002</u>	<u>0.004</u>	<u>-0.019</u>

Evolución por número de componentes



2. Reducción de Dimensiones: PCA/TSNE



- Con esta información, concluimos que la mejor reducción está alrededor de 1/32 de las componentes originales.
- En este punto, no perdemos mucha eficiencia en la clasificación, pero sí ganamos algo muy importante: **el tiempo para ejecutar el entrenamiento es significativamente menor**

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.695</u>	<u>0.720</u>	<u>0.885</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.603</u>	<u>0.537</u>	<u>0.785</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.695</u>	<u>0.720</u>	<u>0.885</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.621</u>	<u>0.604</u>	<u>0.832</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>0.004</u>	<u>0.000</u>	<u>0.000</u>

¿Y el TSNE?

(El mejor obtenido con TSNE: en 1/32)

- En el TSNE, las métricas bajan **considerablemente con respecto al PCA**
- Esto se puede atribuir a que el TSNE **no preserva la estructura global del espacio de características**, solo su distribución
- En nuestro problema, la distribución de los landmarks da lugar a una proyección del TSNE particular (Se verá más adelante) Como la distribución de los landmarks es muy densa y cercana, la hipótesis es que esta nueva distribución complica aún más la clasificación

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Accuracy	<u>0.573</u>	<u>0.586</u>	<u>0.829</u>
train_Precision	<u>0.982</u>	<u>0.982</u>	<u>0.996</u>
test_Precision	<u>0.589</u>	<u>0.541</u>	<u>0.783</u>
train_Recall	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Recall	<u>0.573</u>	<u>0.586</u>	<u>0.829</u>
train_F1_Score	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_F1_Score	<u>0.580</u>	<u>0.561</u>	<u>0.805</u>
train_Cohen_Kappa	<u>0.959</u>	<u>0.965</u>	<u>0.984</u>
test_Cohen_Kappa	<u>-0.026</u>	<u>-0.013</u>	<u>-0.041</u>



3. Reducción de Dimensiones: Otros Algoritmos

- **Truncated SVD**

Técnica de reducción de dimensionalidad que conserva los componentes más importantes sin centrar los datos.

- Reduce dimensiones como PCA, pero es mejor para matrices dispersas (como TF-IDF).
- No necesita centrar, funciona con datos dispersos y es más eficiente en gran escala.

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.692</u>	<u>0.720</u>	<u>0.886</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.593</u>	<u>0.528</u>	<u>0.785</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.692</u>	<u>0.720</u>	<u>0.886</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.616</u>	<u>0.604</u>	<u>0.832</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>-0.009</u>	<u>0.001</u>	<u>-0.000</u>

3. Reducción de Dimensiones: Otros Algoritmos

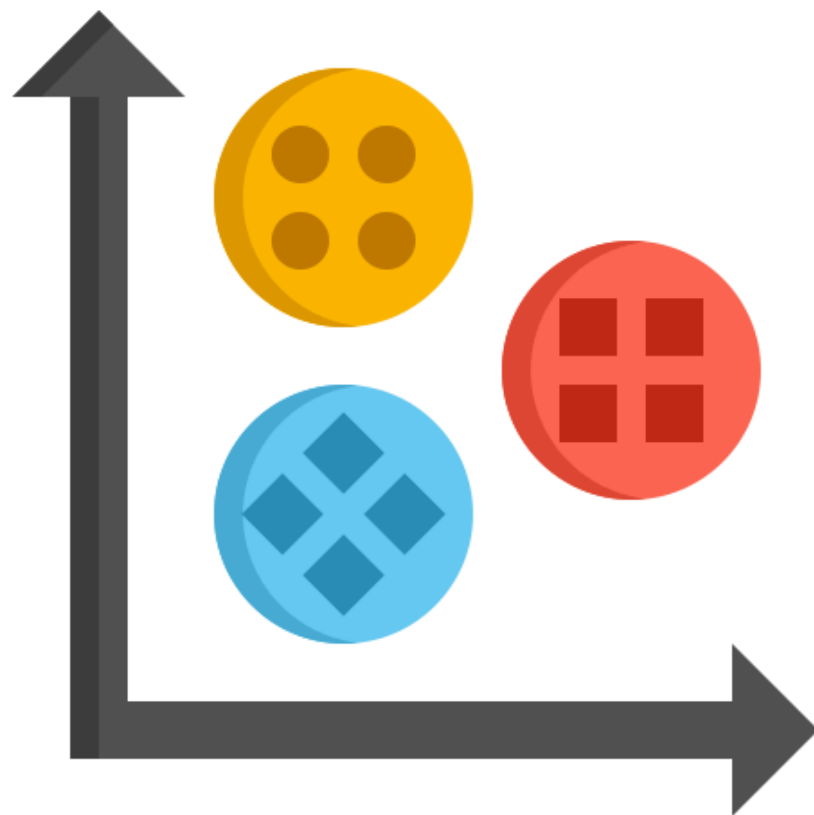
- **Factor Analysis**

Técnica de reducción que modela la variabilidad común entre variables como factores latentes.

- A diferencia de PCA, se enfoca en las relaciones ocultas, no en la varianza total.
- Útil para identificar estructuras subyacentes, pero menos eficiente con datos dispersos.

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.702</u>	<u>0.718</u>	<u>0.886</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.605</u>	<u>0.535</u>	<u>0.785</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.702</u>	<u>0.718</u>	<u>0.886</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.620</u>	<u>0.604</u>	<u>0.832</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>0.006</u>	<u>-0.001</u>	<u>-0.001</u>

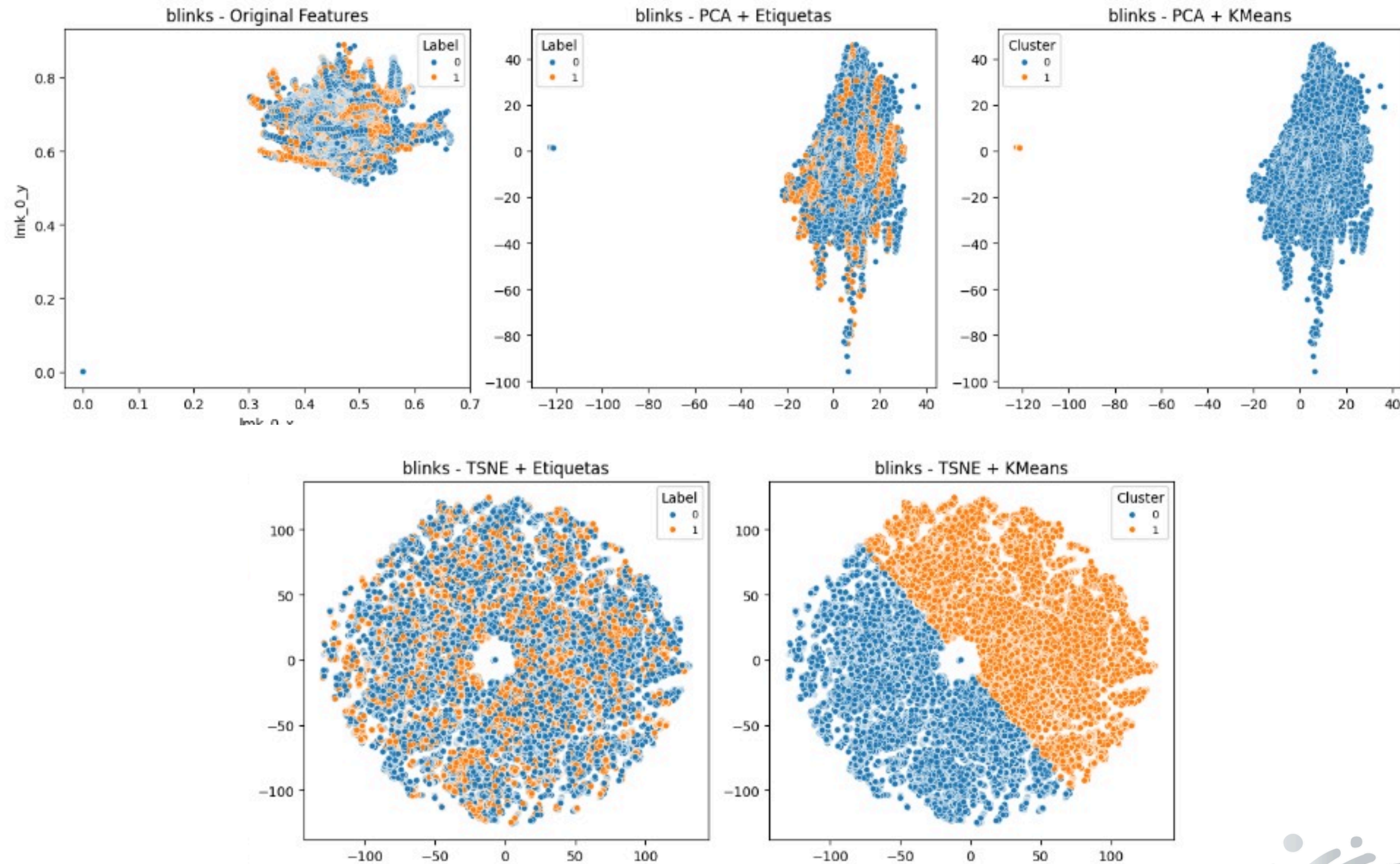
4. Métodos de Agrupamiento: K-Means, DBSCAN y Aglomerative Clustering



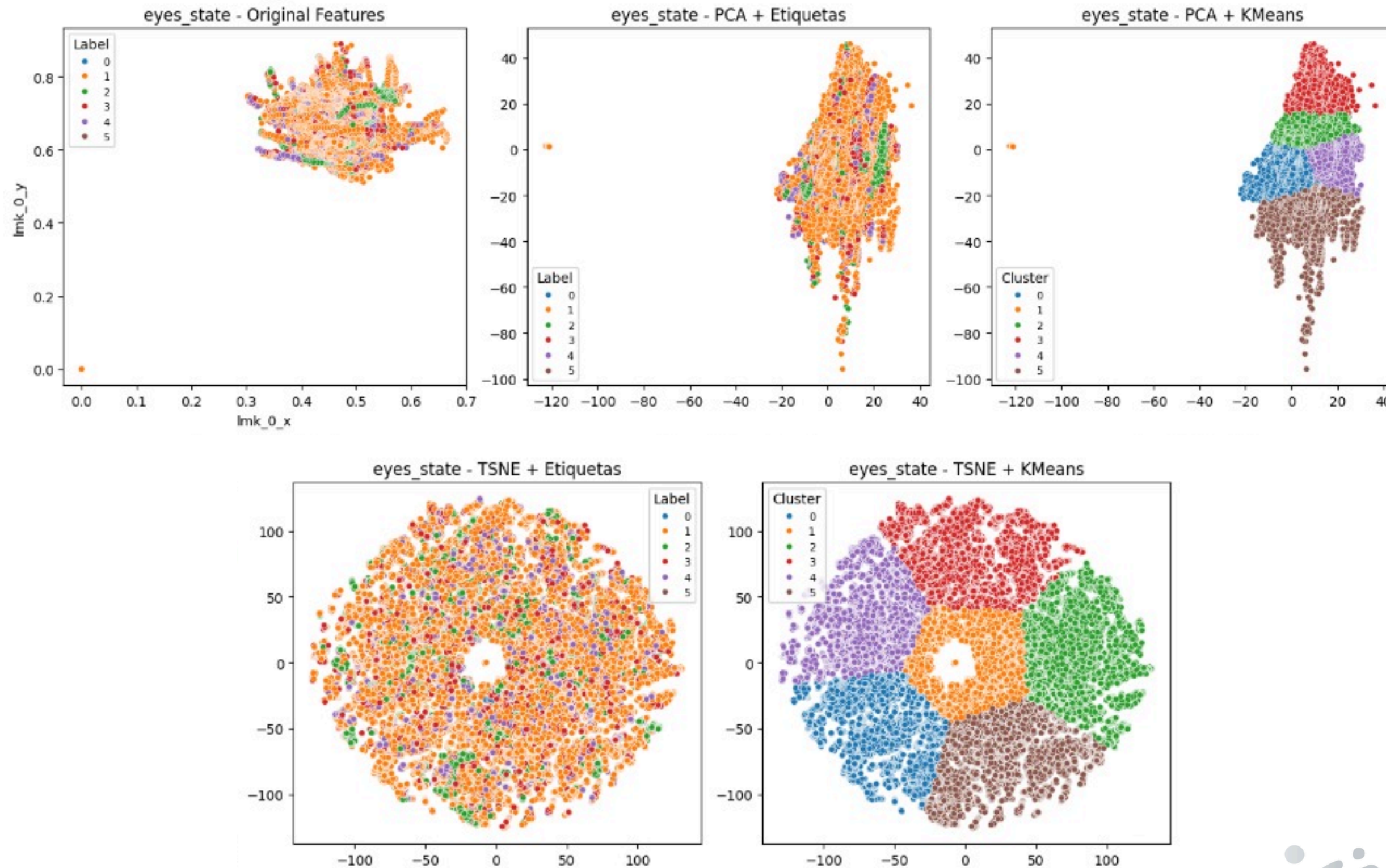
- Ahora, vamos a intentar encontrar si, tras aplicar técnicas de reducción de componentes, **los algoritmos de clustering son capaces de diferenciar correctamente los grupos del dataset**
- Por ello, vamos a partir de una representación visual en **2 componentes**: una generada por un PCA, y otro por un TSNE

4.1 K-Means

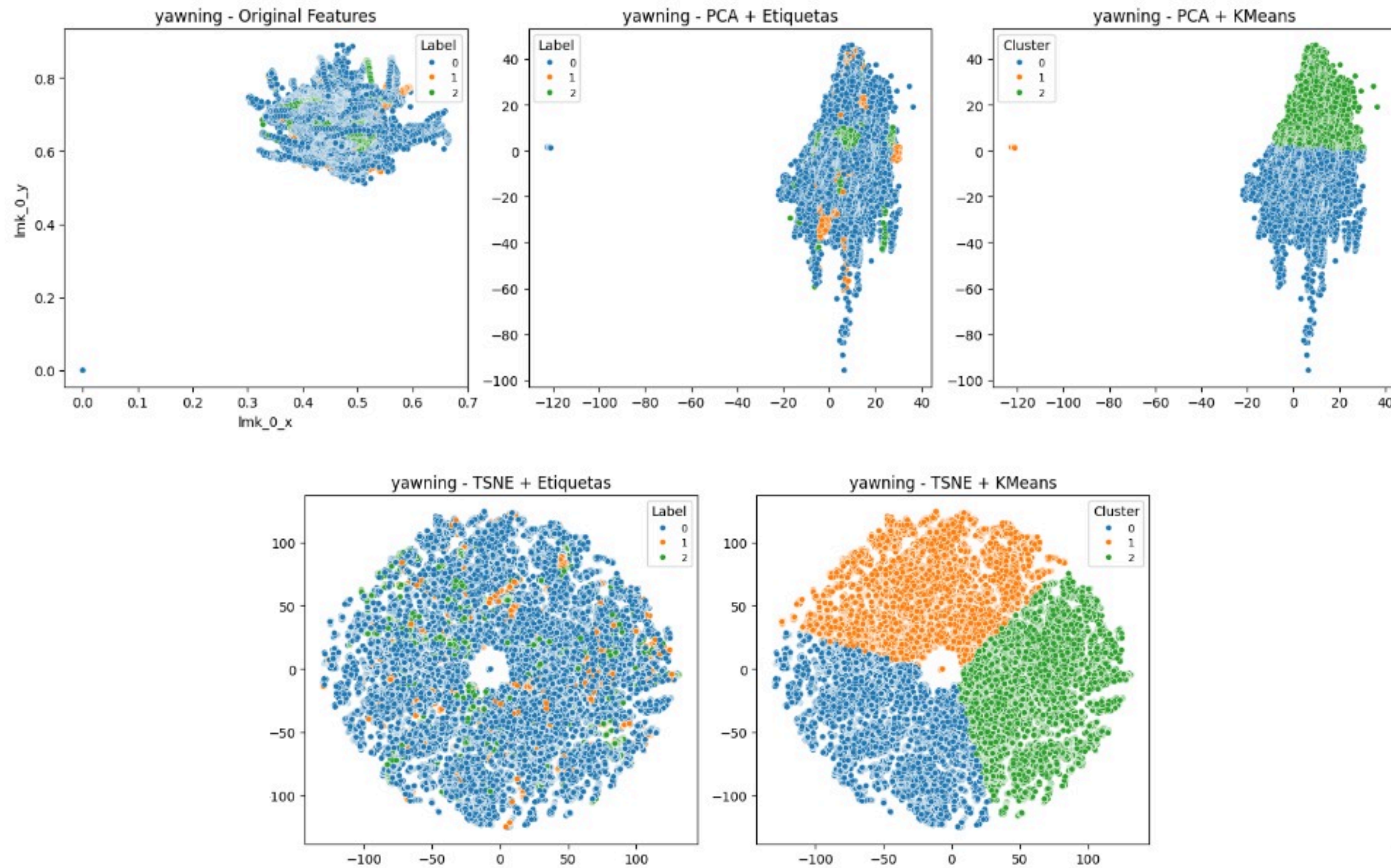
Comparación de Clustering por Target



4.1 K-Means

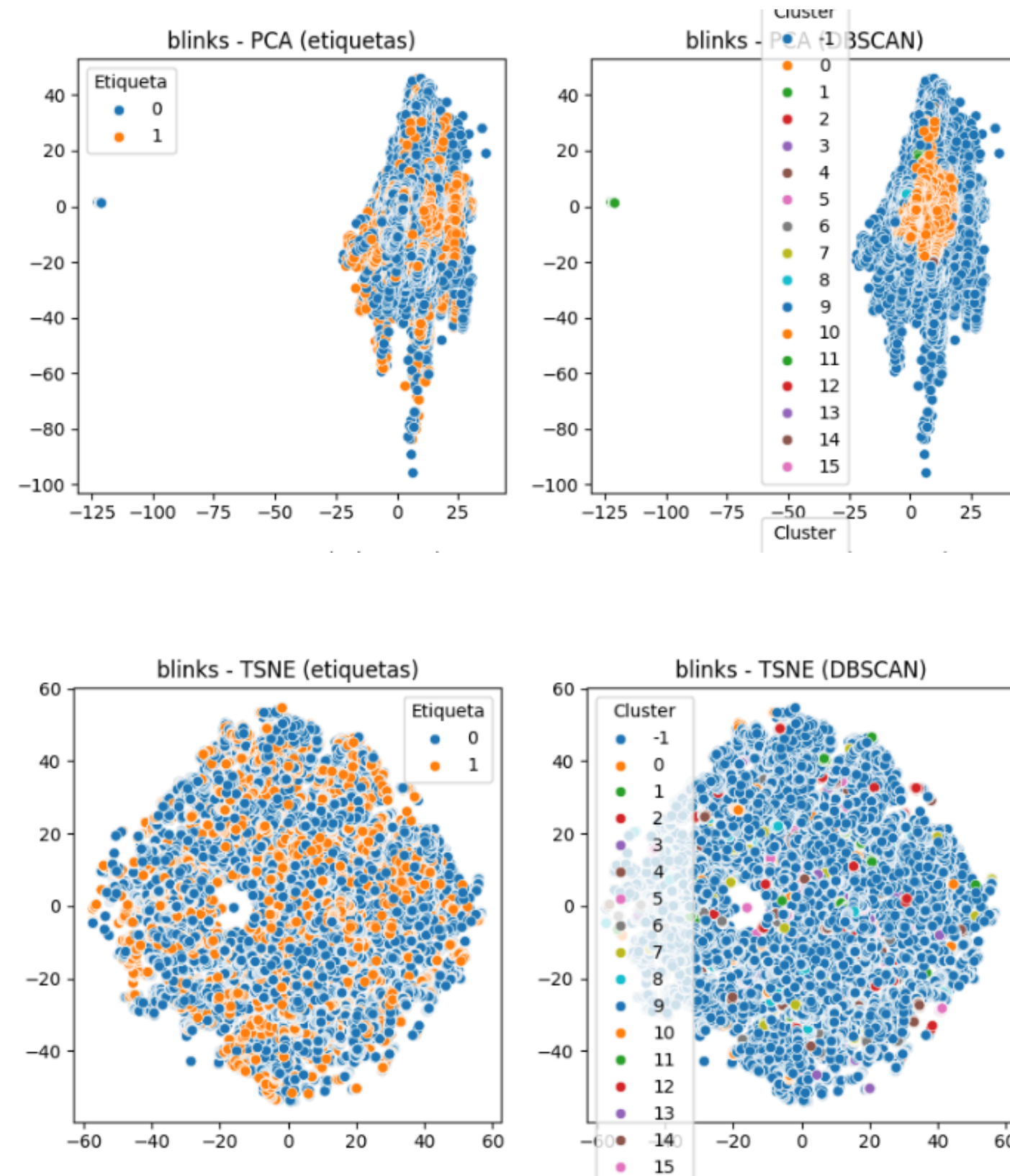


4.1 K-Means



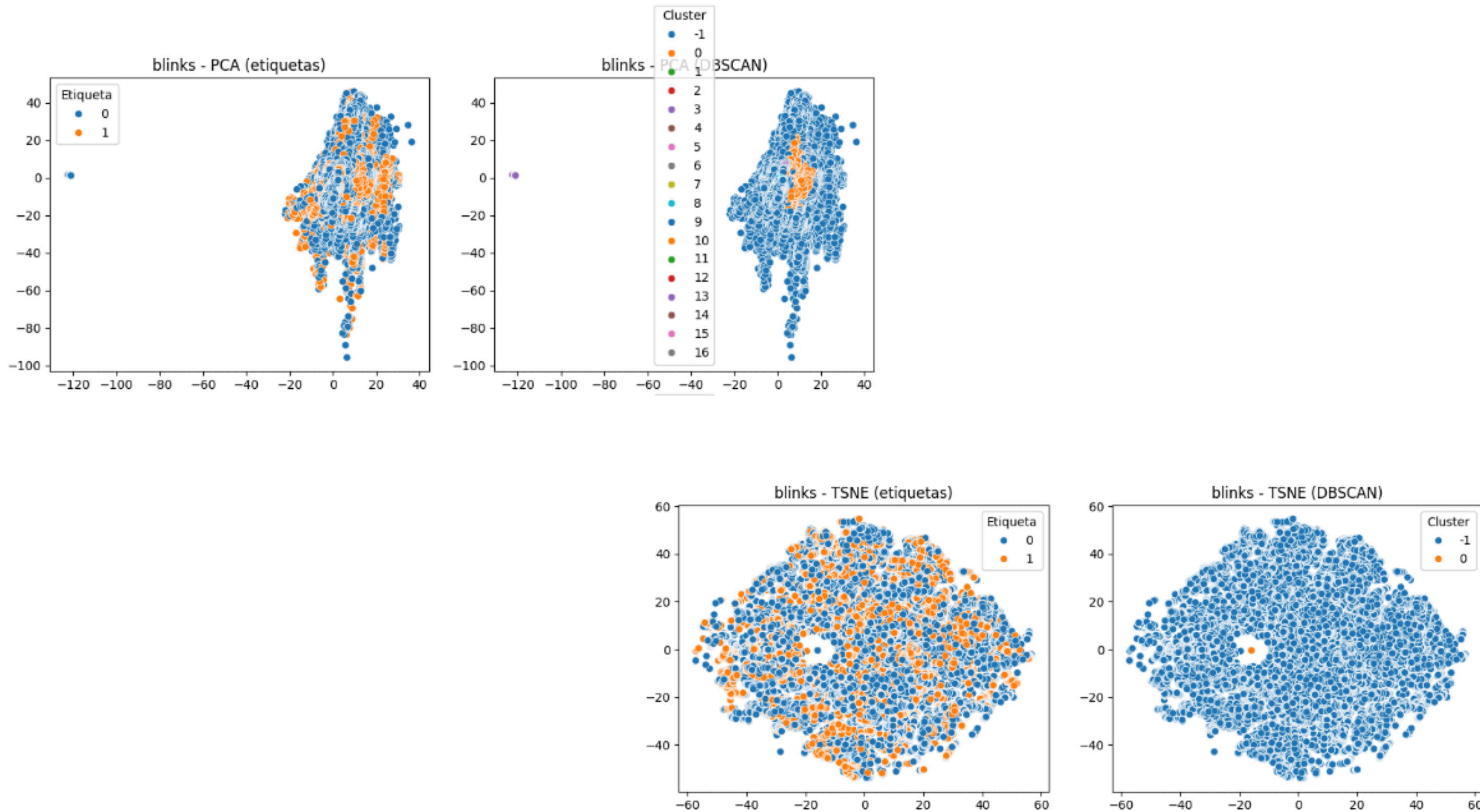
4.2 DBSCAN

- Con DBSCAN, los resultados son muy variables dependiendo de los hiperpárametros. Se probó con variaciones de ϵ y min_samples , pero los resultados de separación son genuinamente irrelevantes.
- (Con un ejemplo de $\text{min_samples} = 50$), $\epsilon = 0.5$



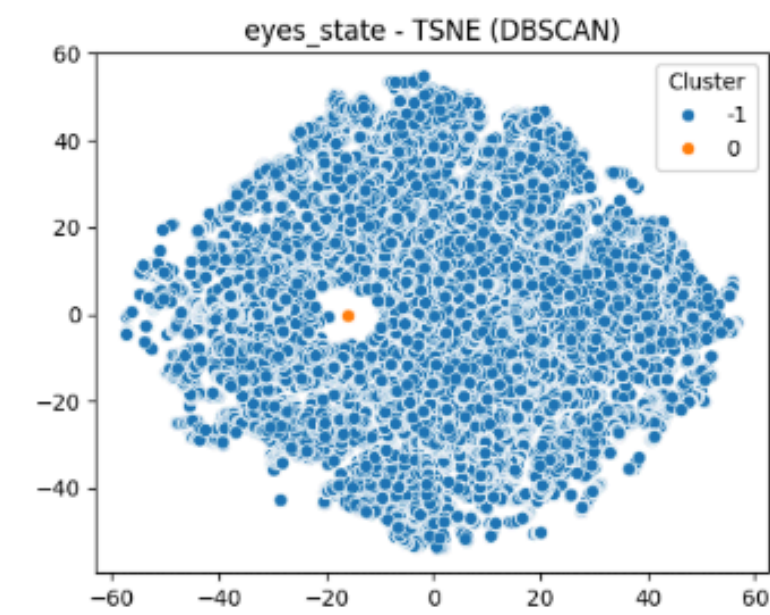
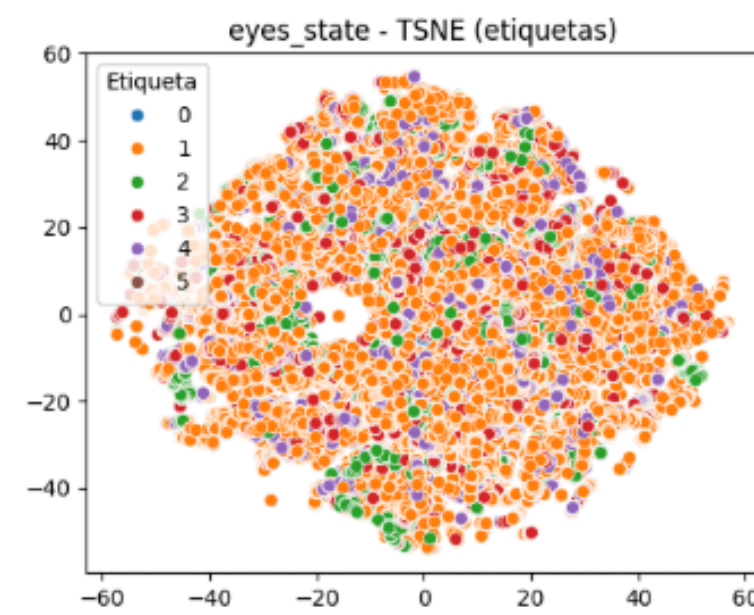
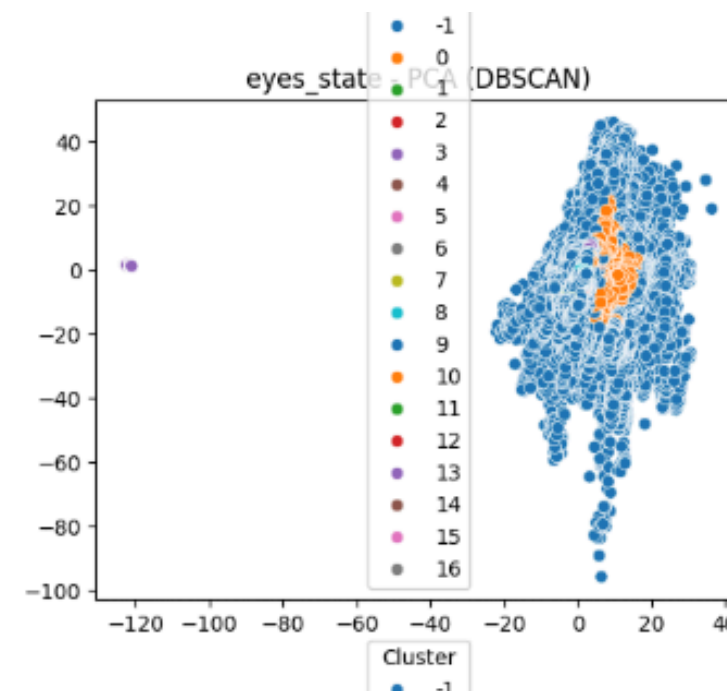
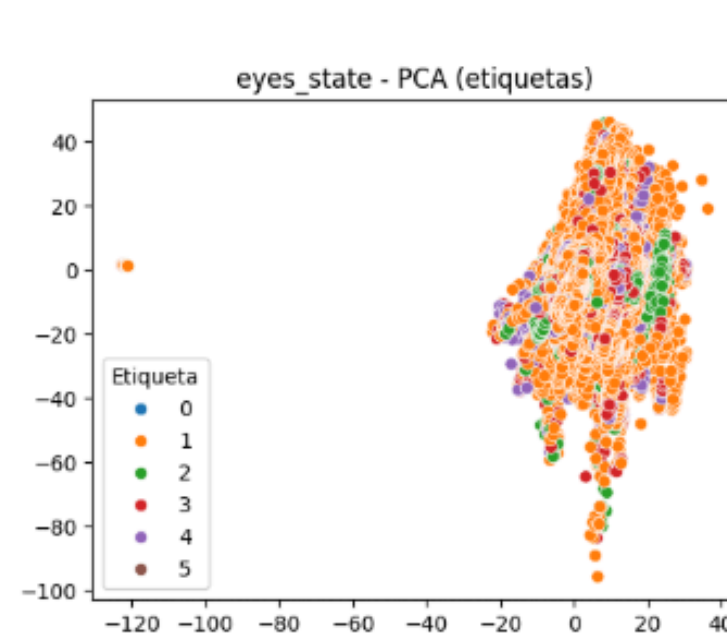
4.2 DBSCAN

- (Con un ejemplo de $\text{min_samples} = 100$), $\epsilon = 0.5$



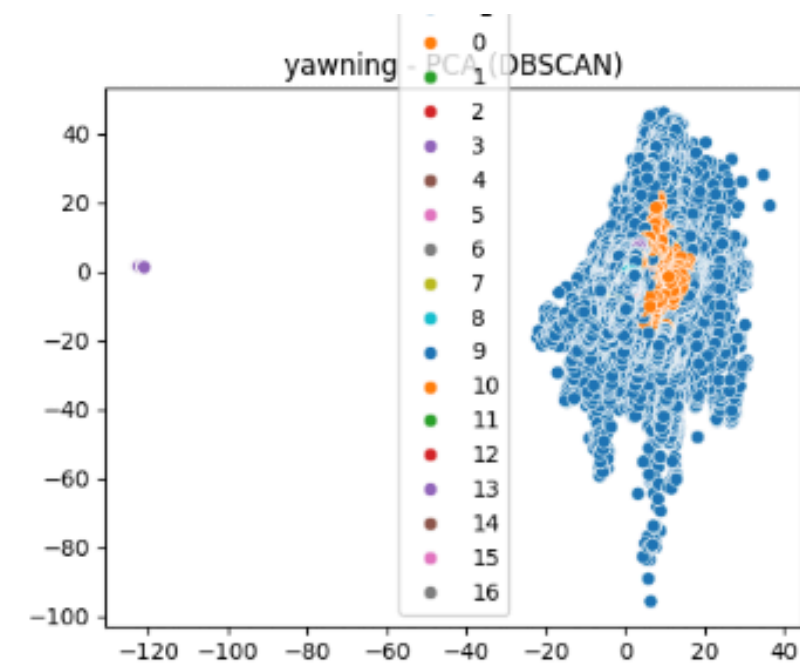
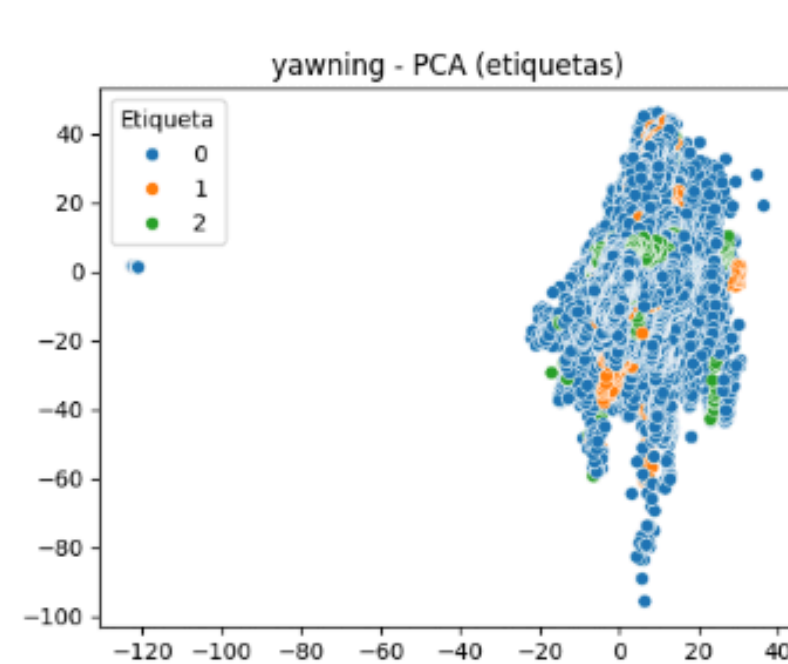
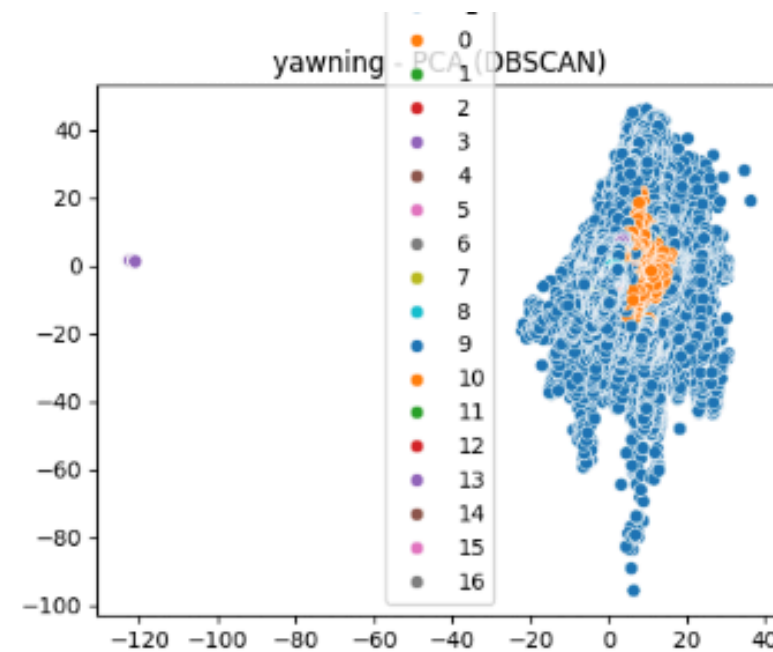
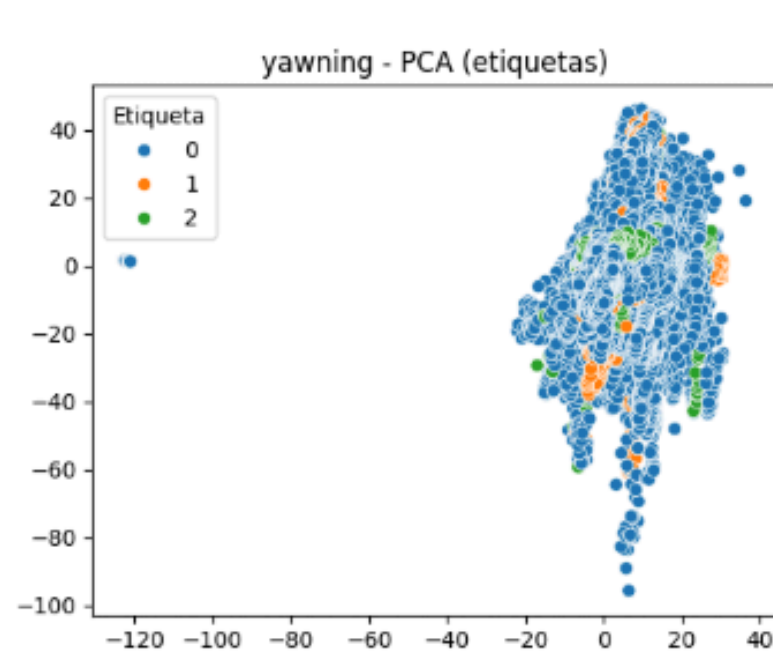
4.2 DBSCAN

- (Con un ejemplo de $\text{min_samples} = 100$), $\epsilon = 0.5$



4.2 DBSCAN

- (Con un ejemplo de $\text{min_samples} = 100$, $\epsilon = 0.5$)

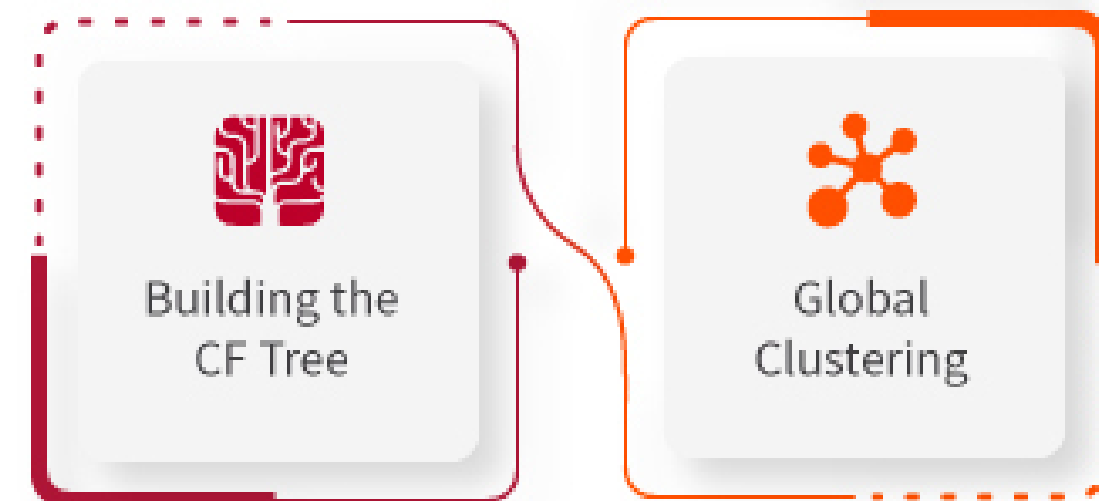


5. Otros algoritmos de clustering

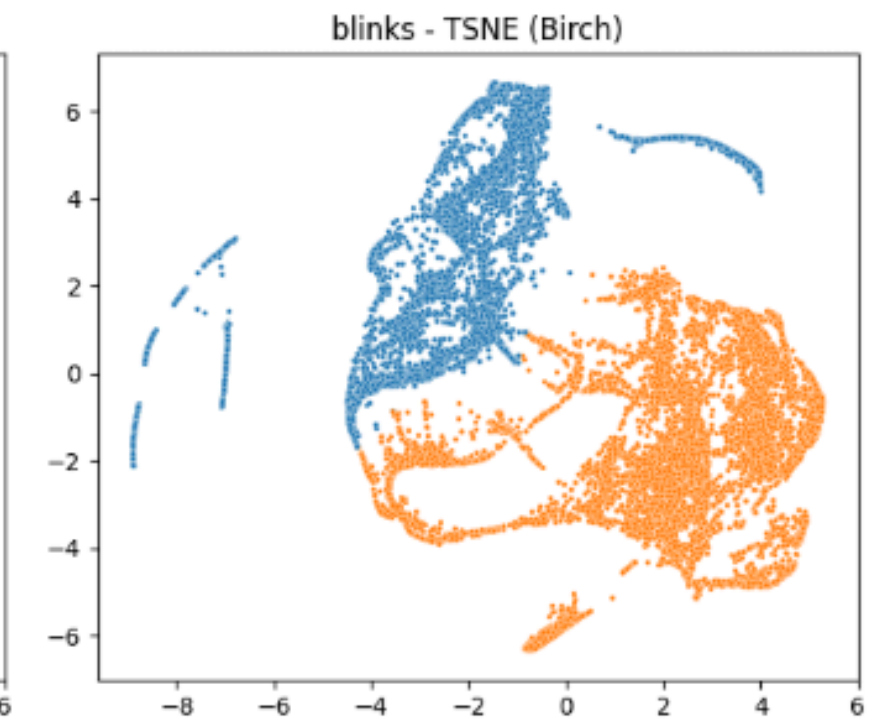
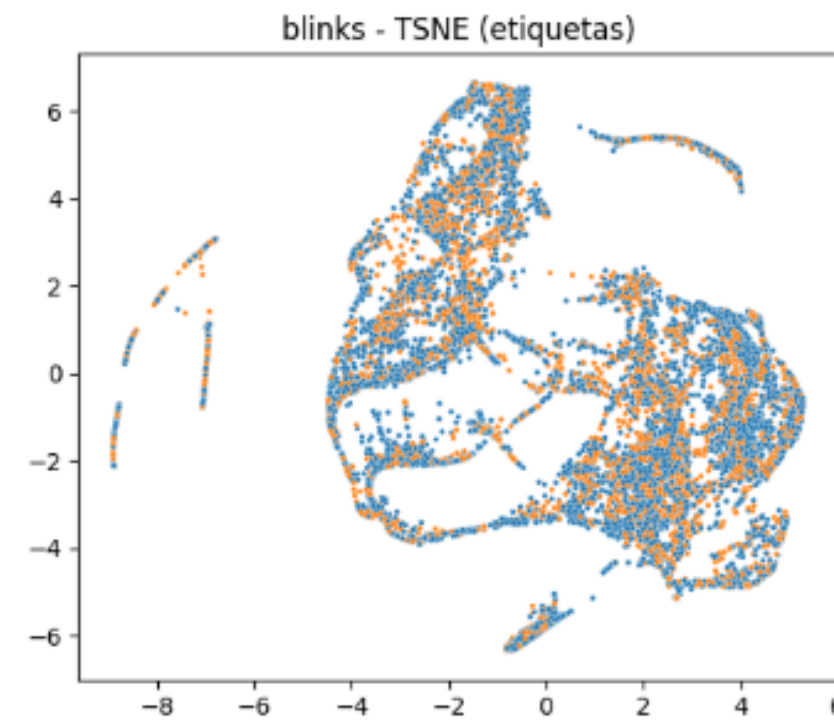
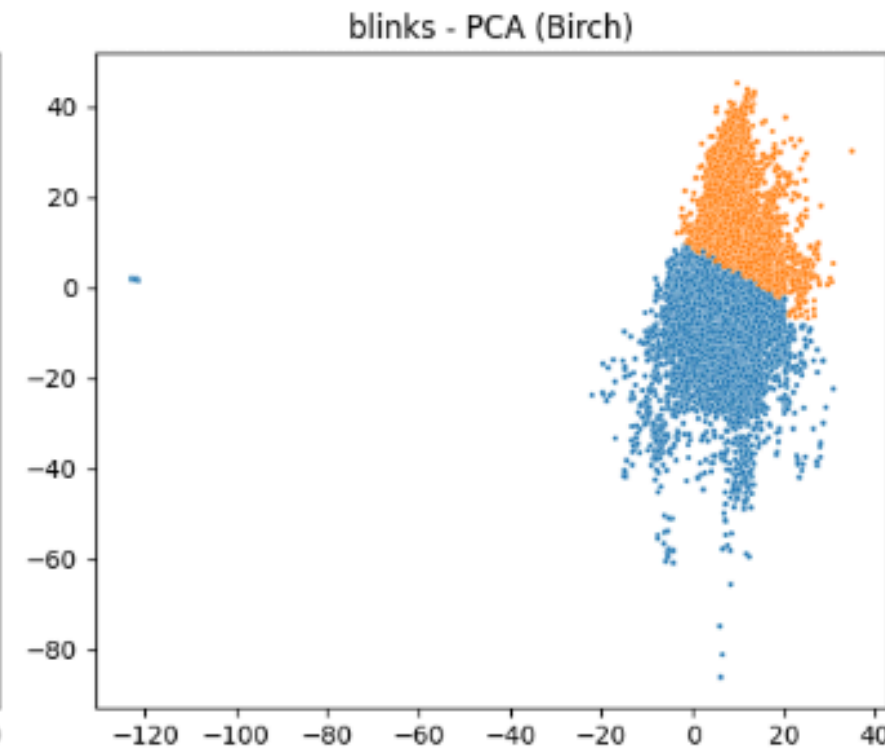
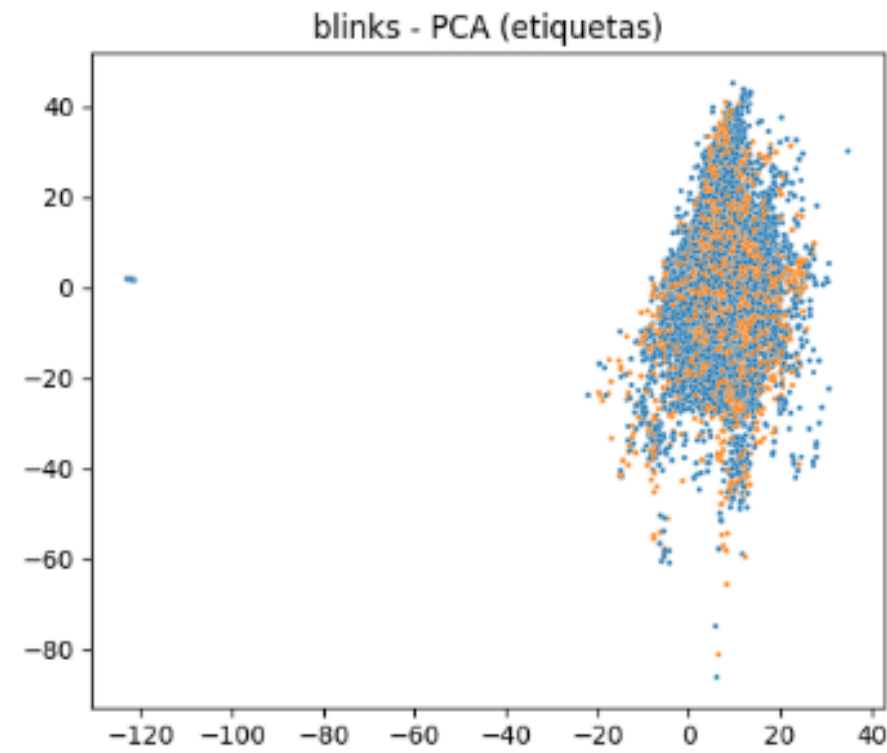
5.1 - BIRCH

- Construye un árbol de características (**CF Tree**) que resume los datos en pequeñas subestructuras llamadas clústeres subyacentes.
- Agrupa datos de forma jerárquica y eficiente
- Es bueno para grandes cantidades de datos por esa misma razón.

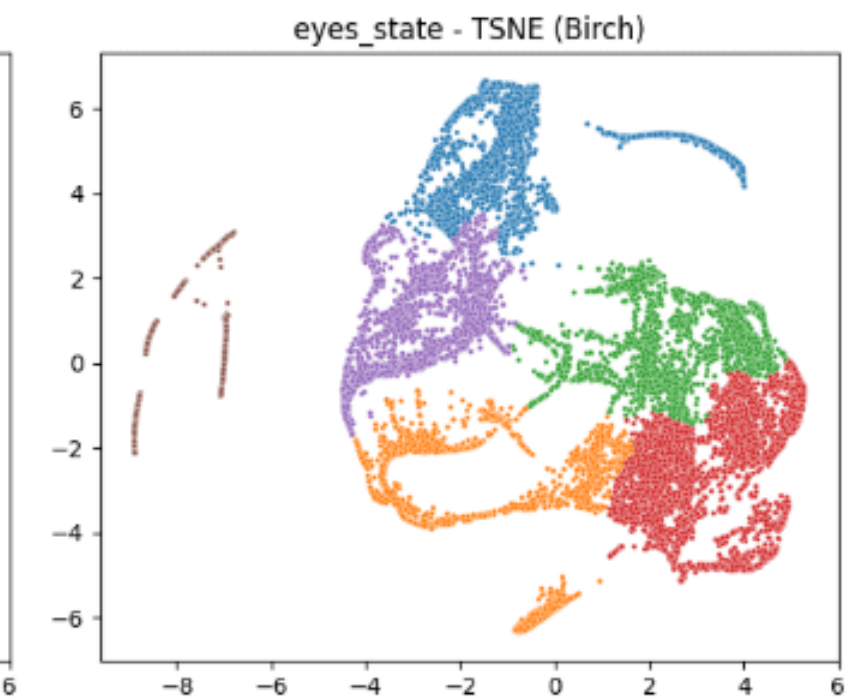
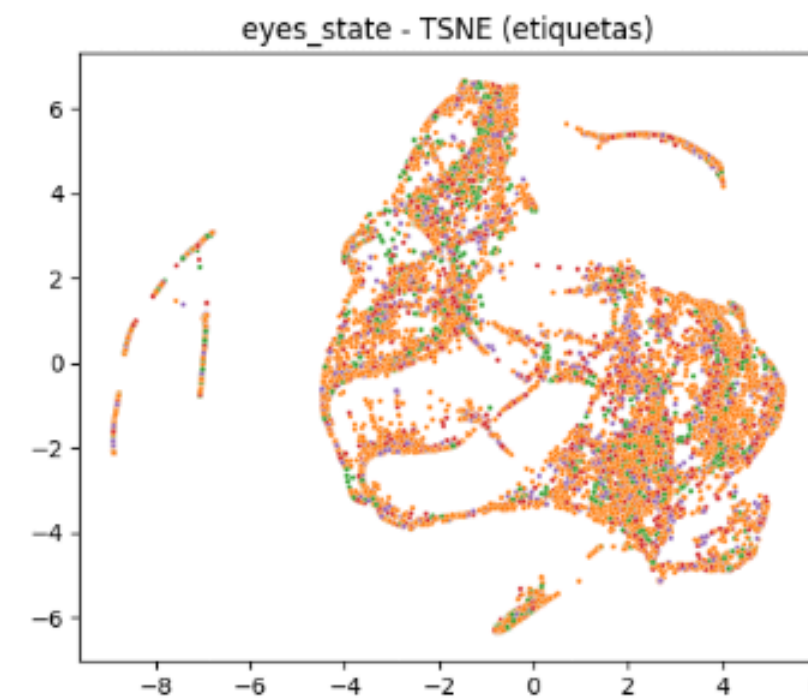
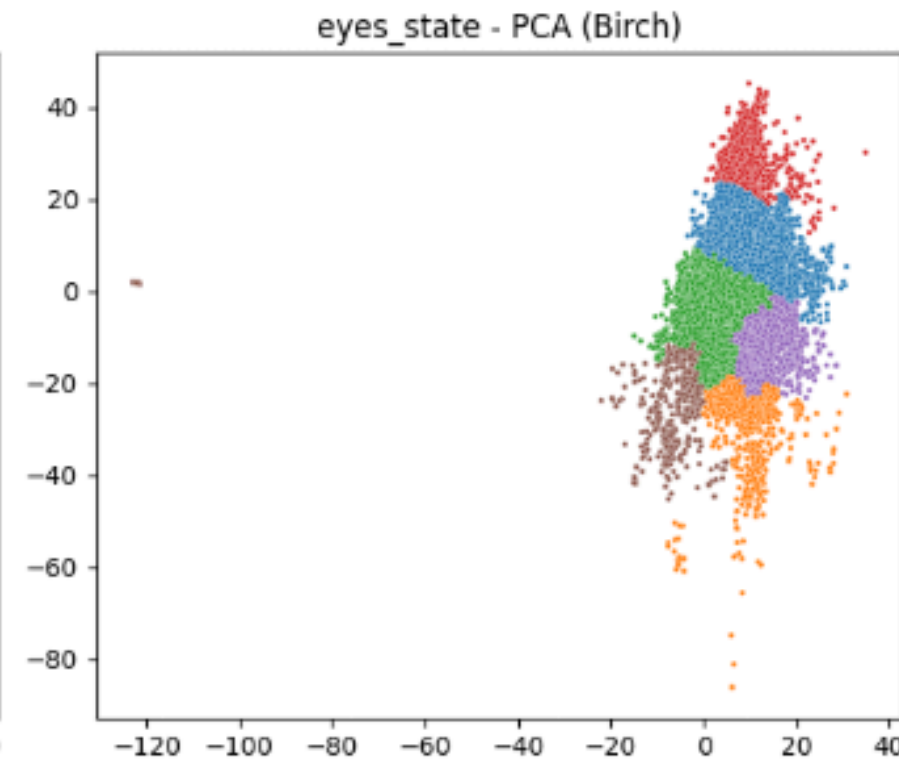
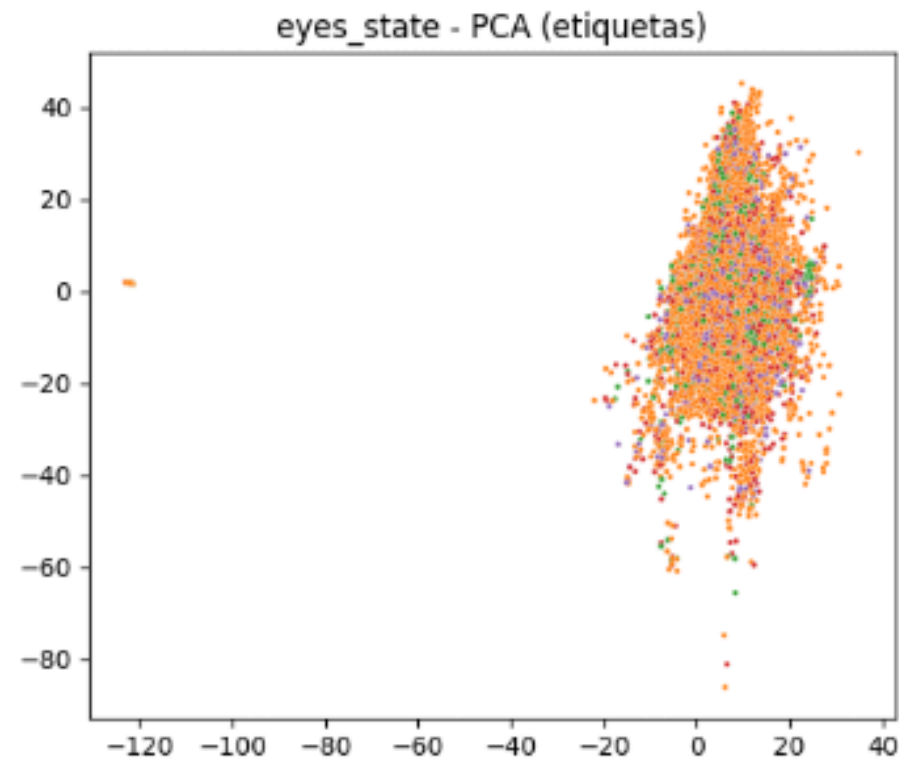
The BIRCH clustering algorithm consists of two stages:



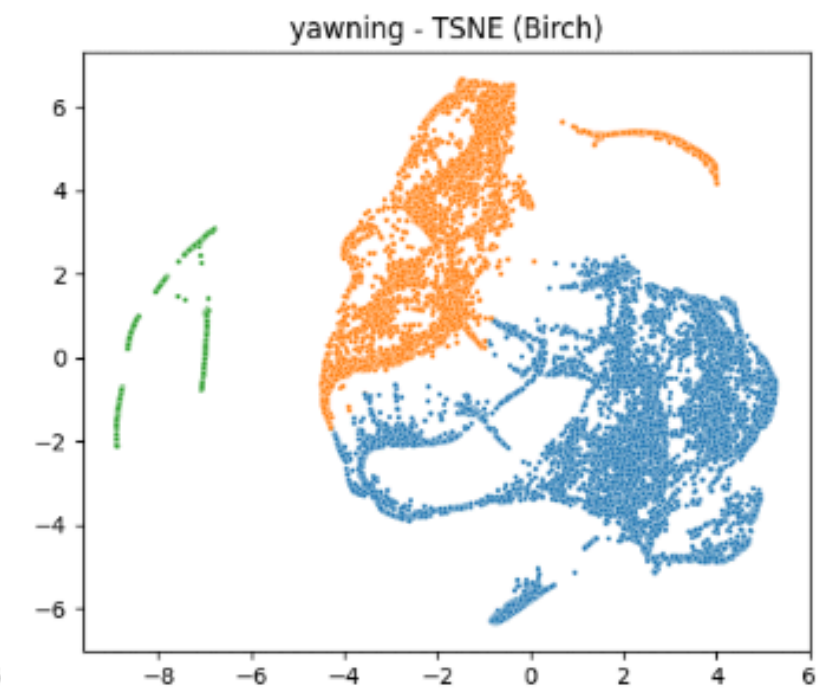
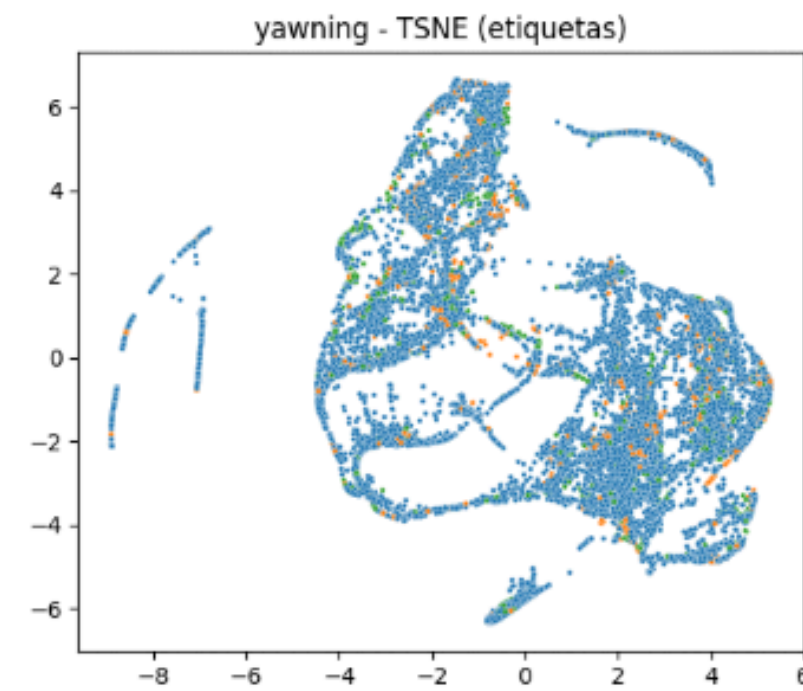
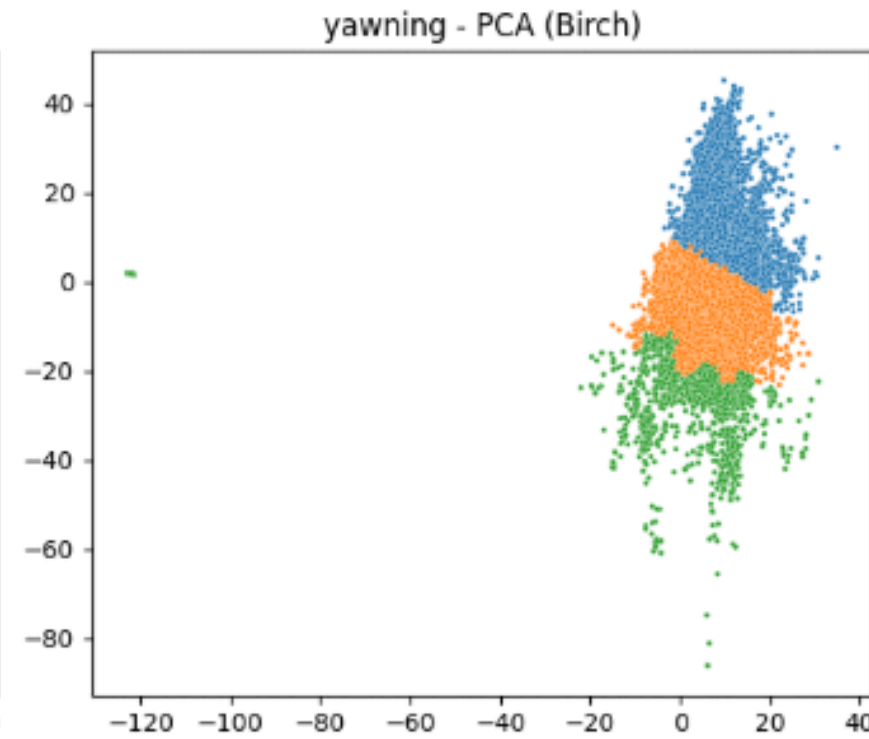
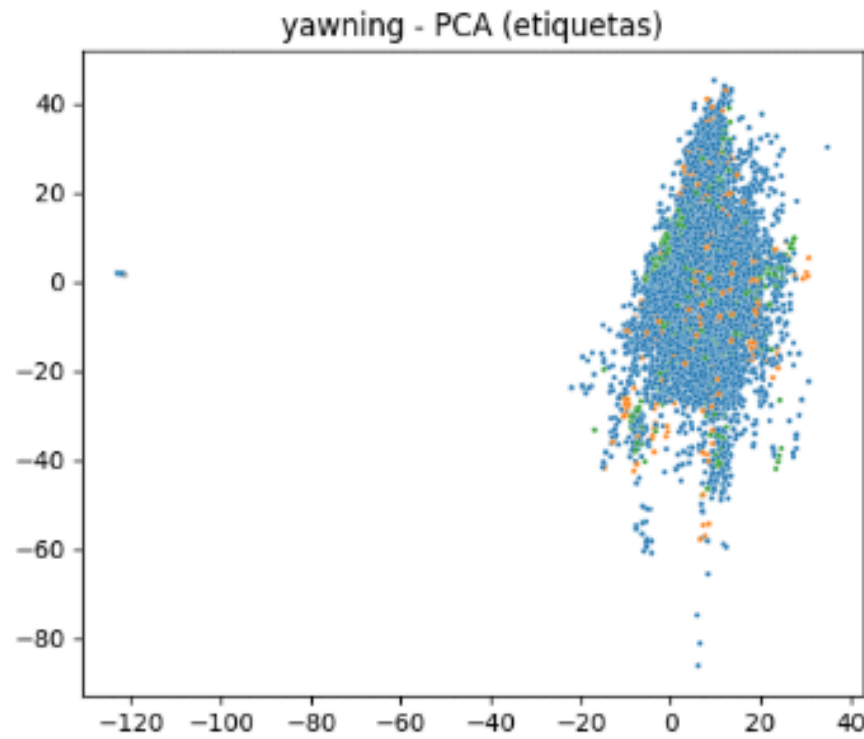
5.1 - BIRCH



5.1 - BIRCH



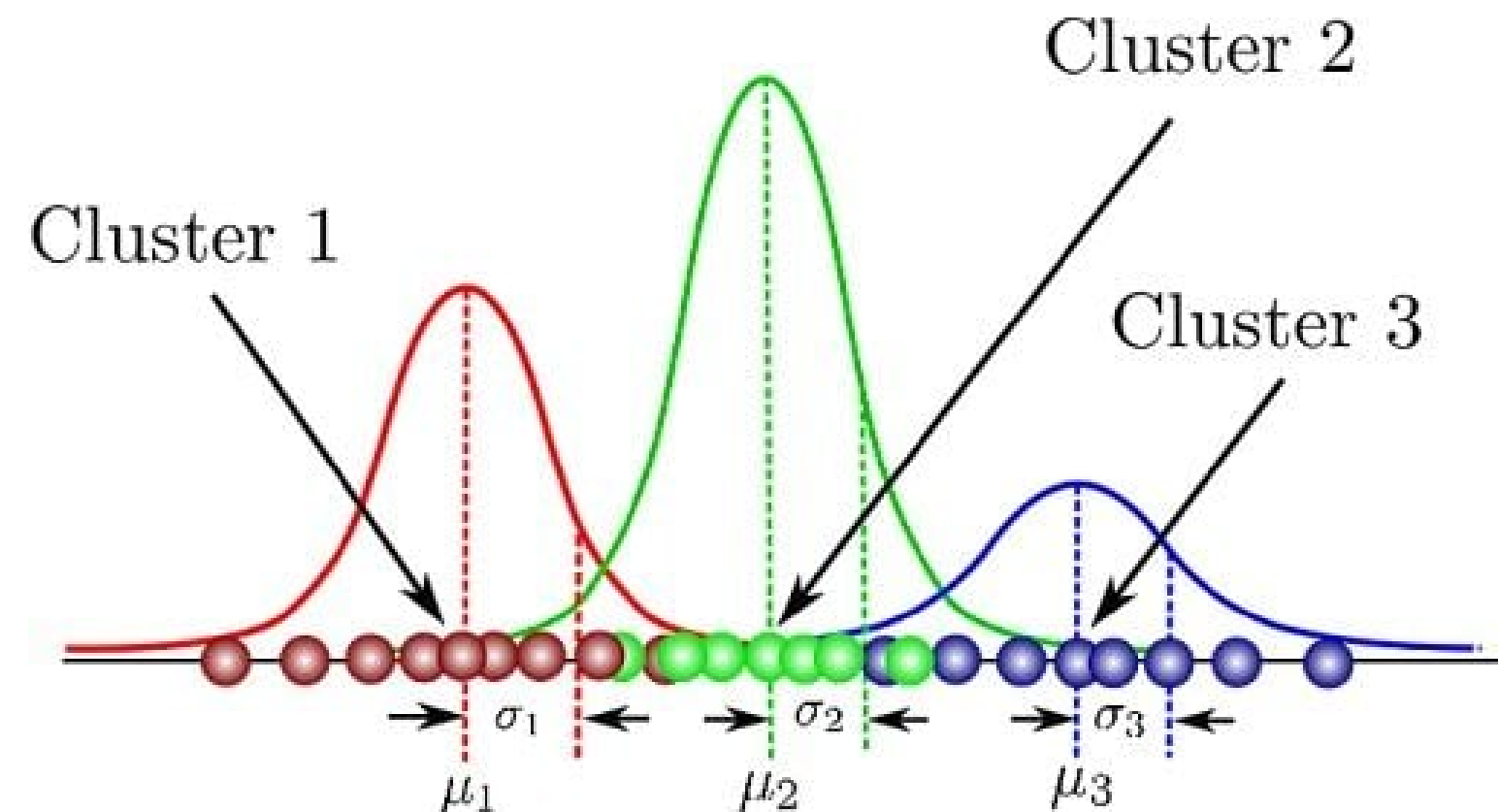
5.1 - BIRCH



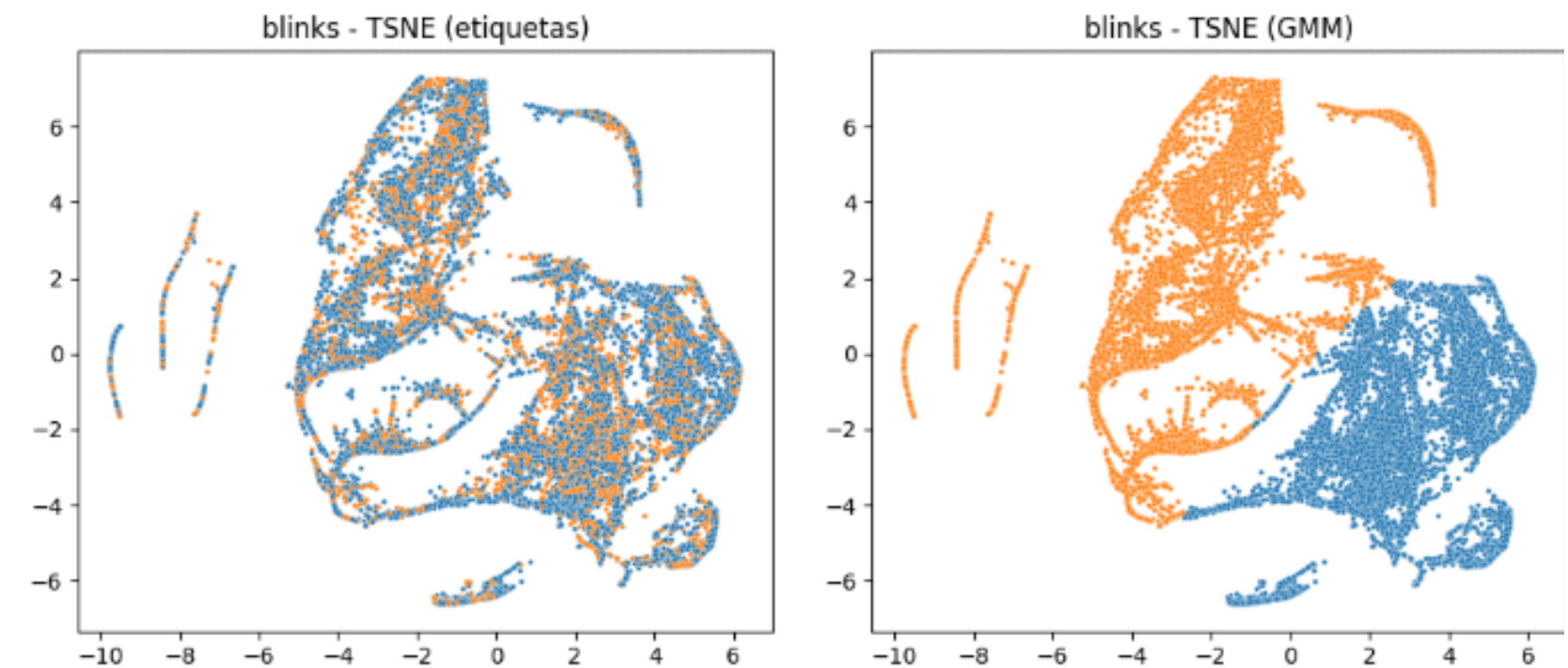
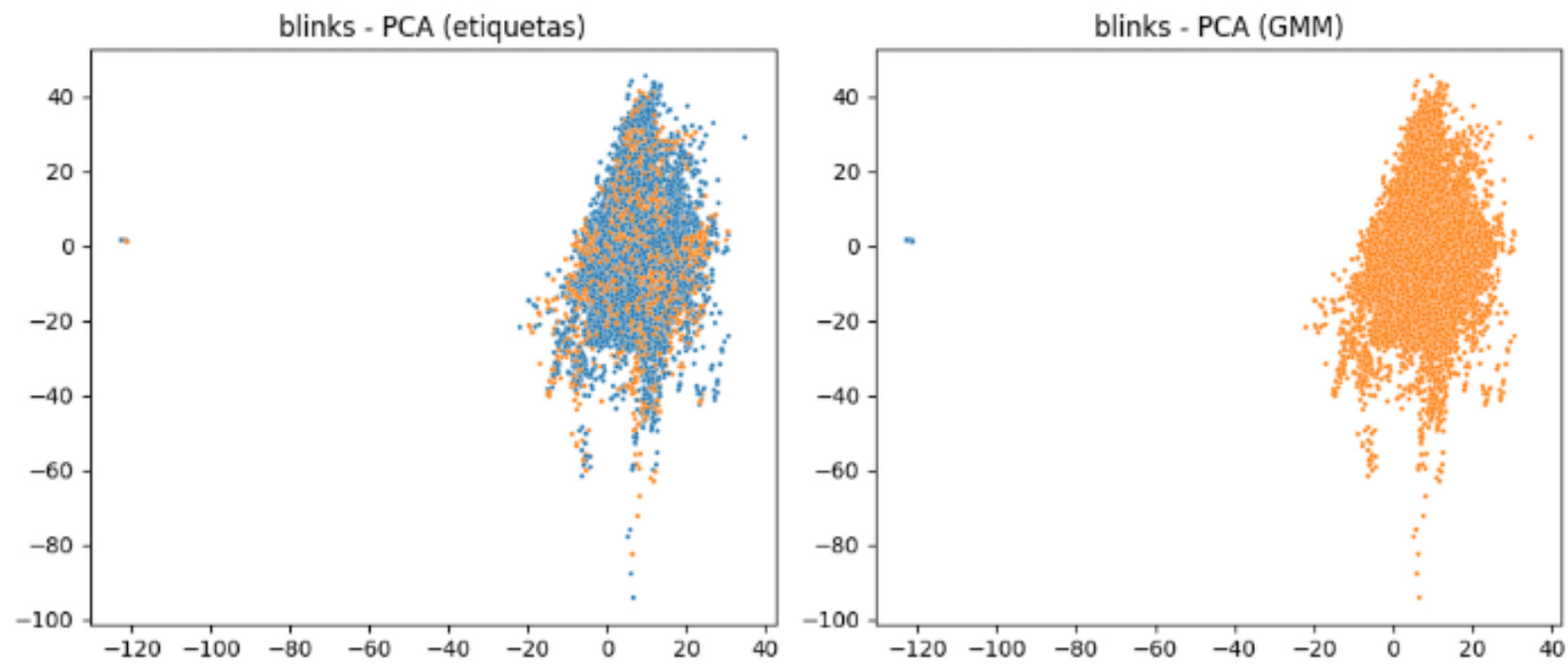
5. Otros algoritmos de clustering

5.2 - Gaussian Mixture

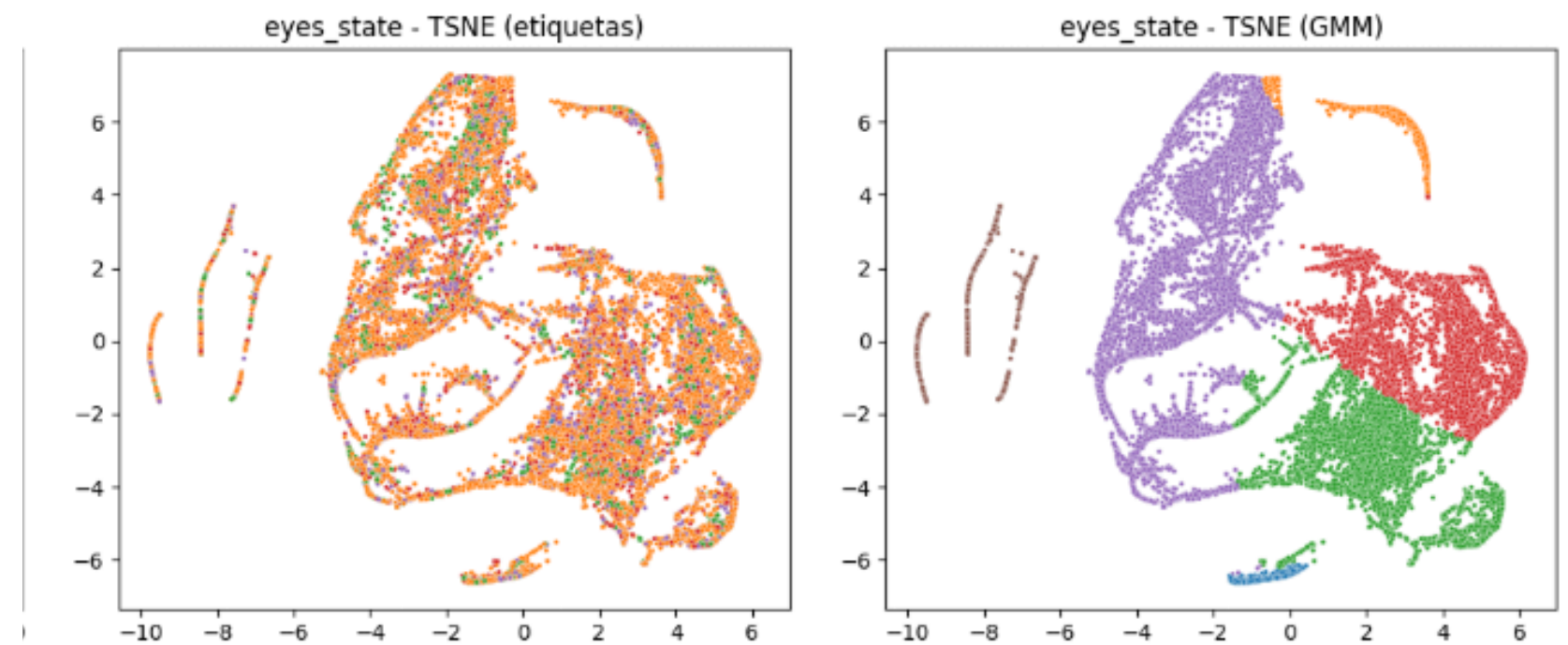
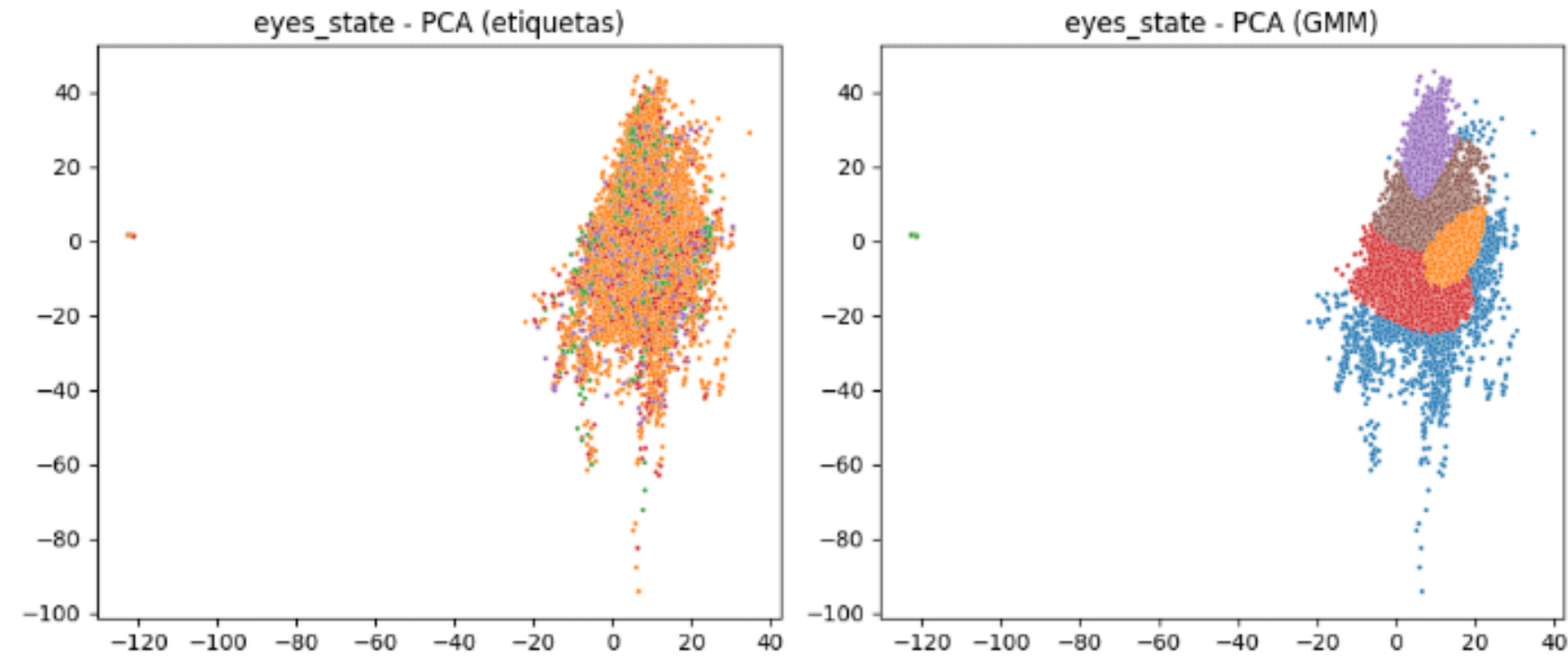
- Se basa en modelos estadísticos, asumiendo que los datos provienen de una combinación de varias distribuciones normales (gaussianas).
- Es probabilístico, lo que significa que asigna a cada punto una probabilidad de pertenecer a cada clúster.
- Se itera con el algoritmo **Expectation-Maximization**



5.2 - Gaussian Mixture

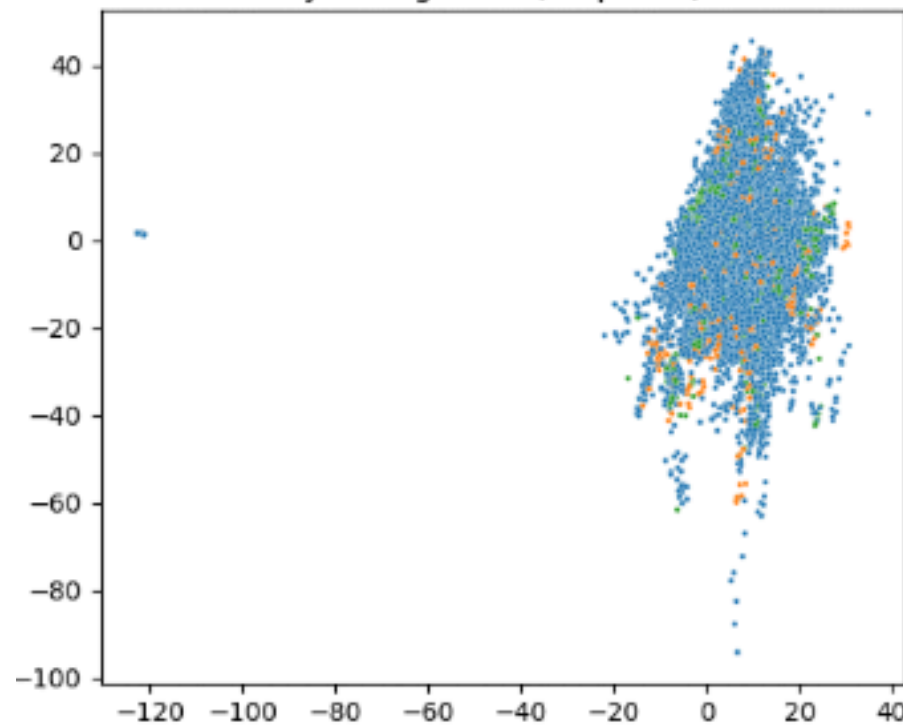


5.2 - Gaussian Mixture

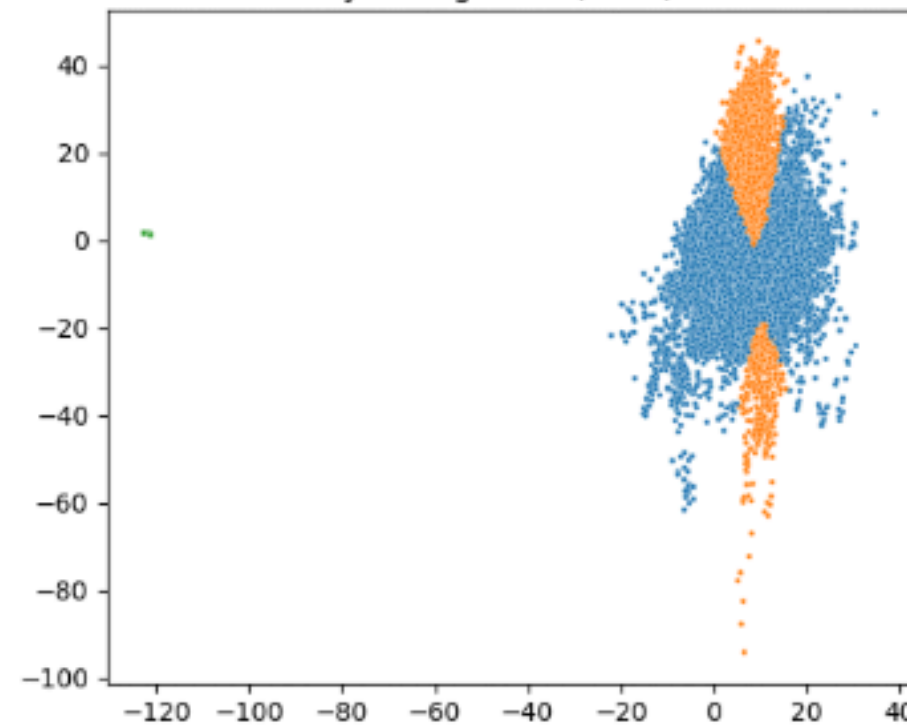


5.2 - Gaussian Mixture

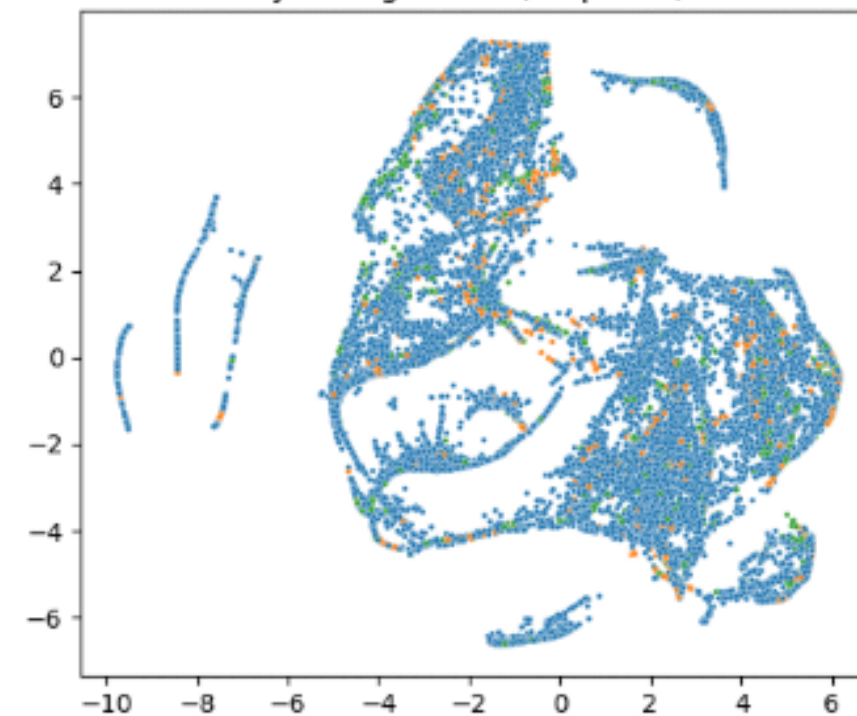
yawning - PCA (etiquetas)



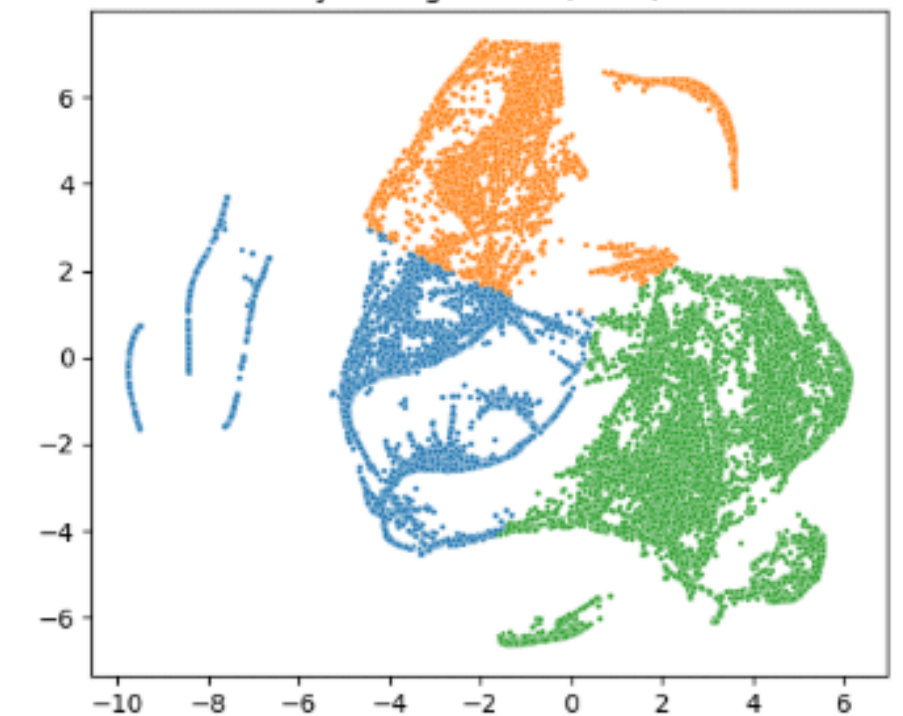
yawning - PCA (GMM)



yawning - TSNE (etiquetas)

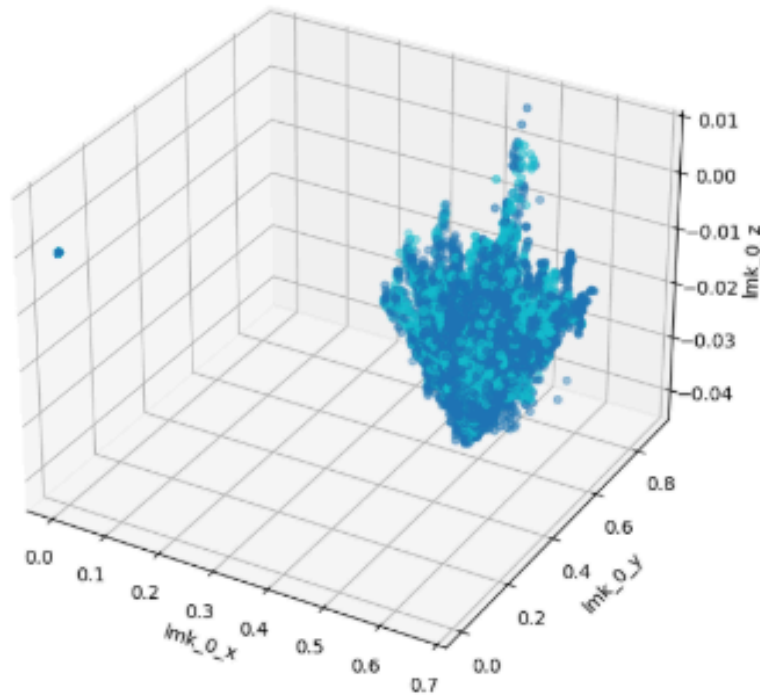


yawning - TSNE (GMM)

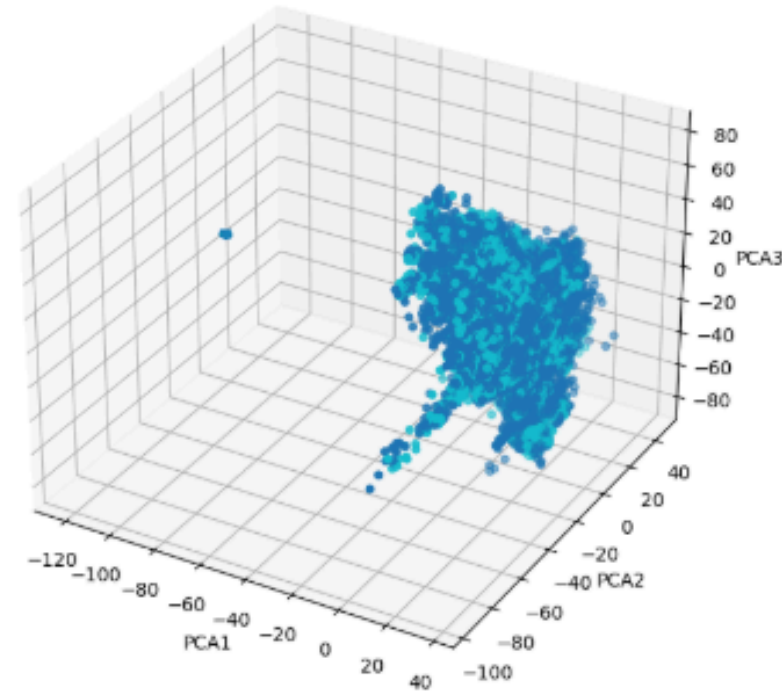


5.3 - Bonus: K-Means en 3D

blinks - Original Features

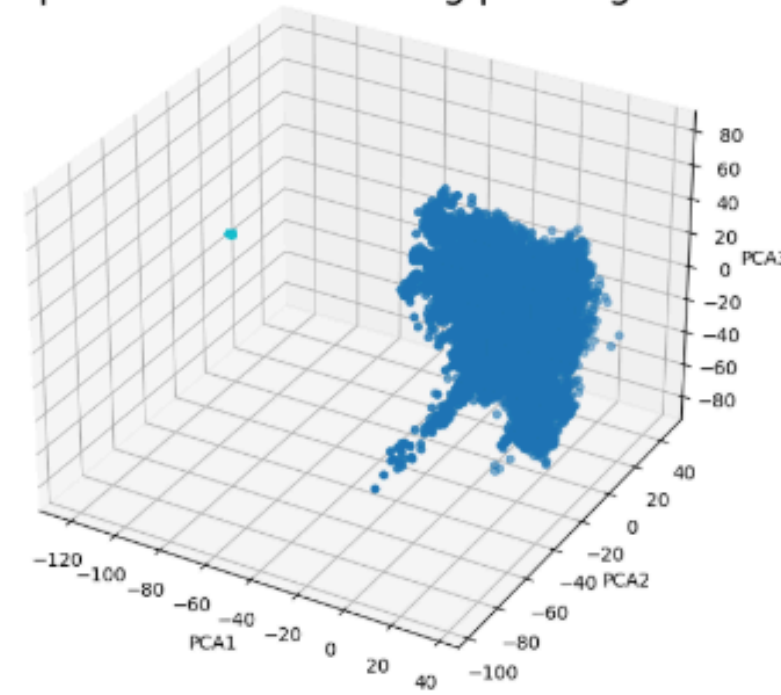


blinks - PCA + Etiquetas

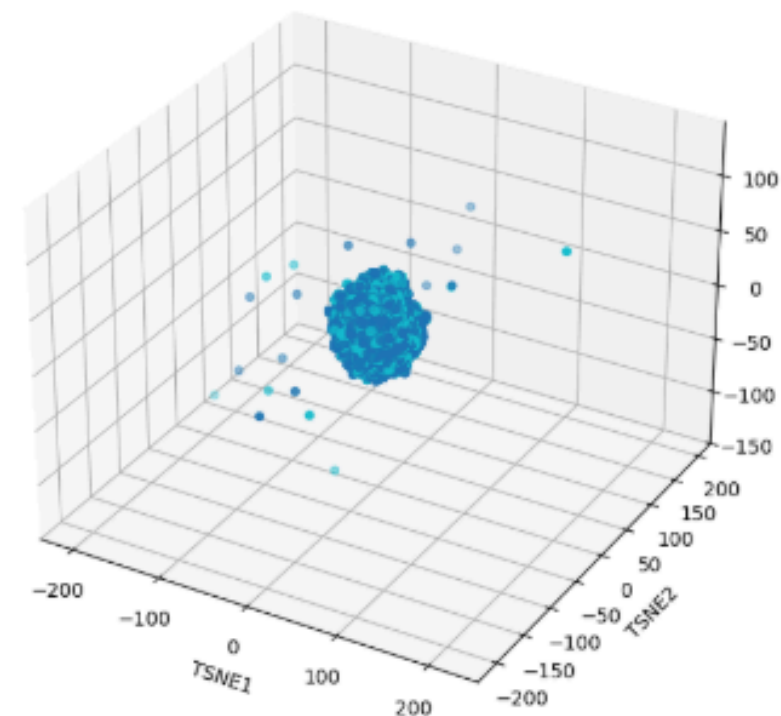


blinks - PCA + KMeans

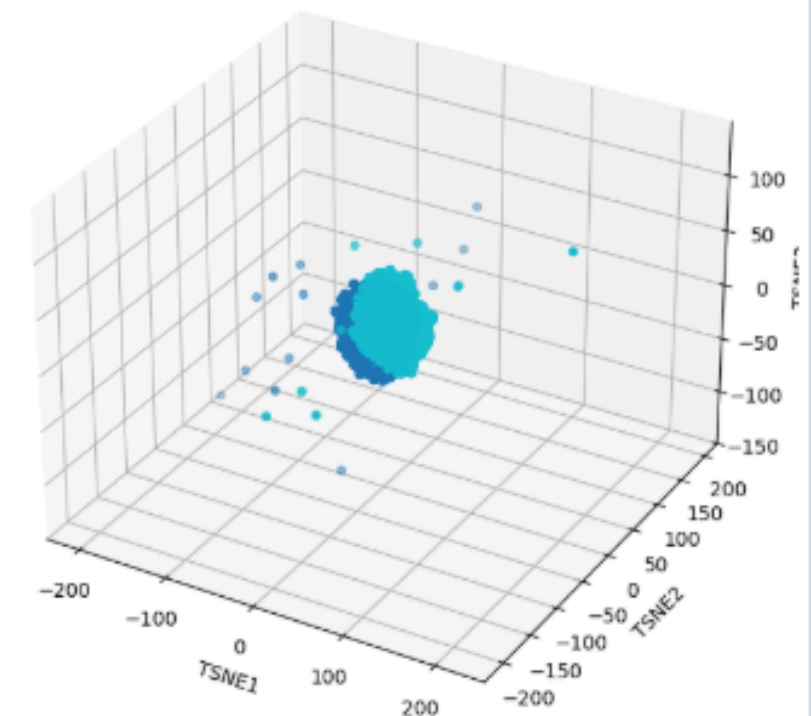
Comparación de Clustering por Target - 3D



blinks - TSNE + Etiquetas

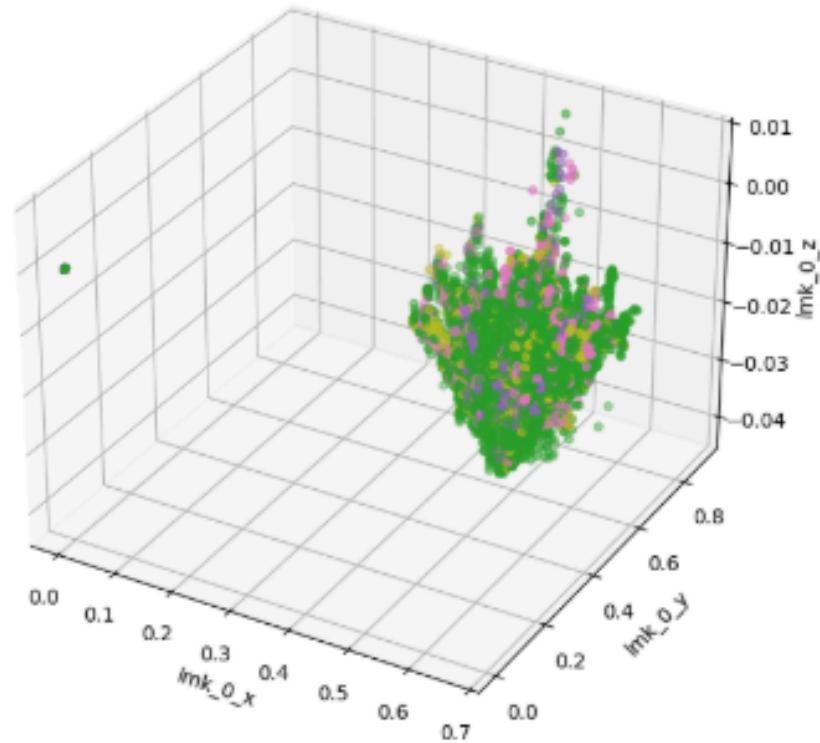


blinks - TSNE + KMeans

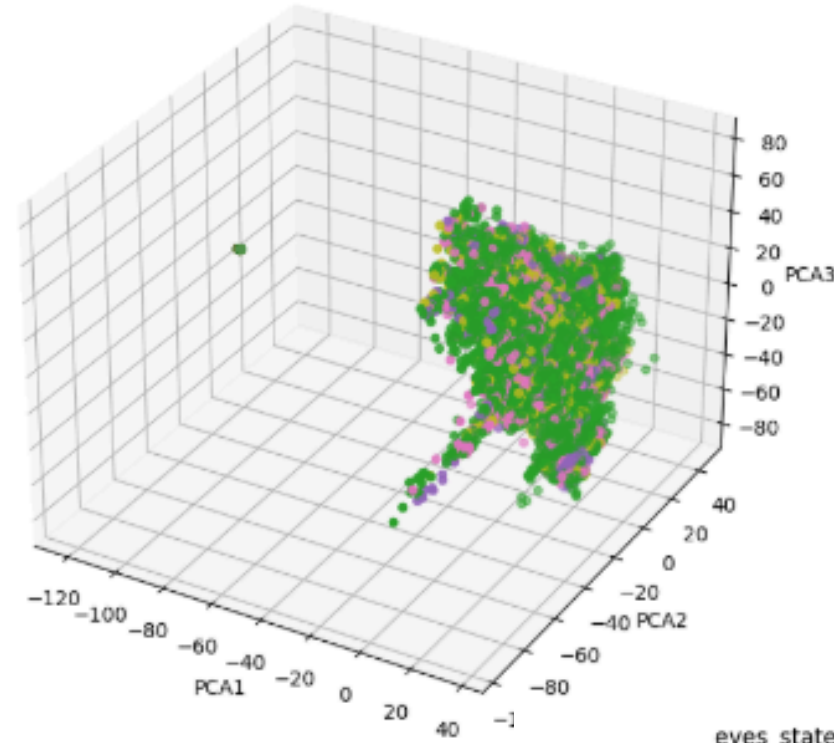


5.3 - Bonus: K-Means en 3D

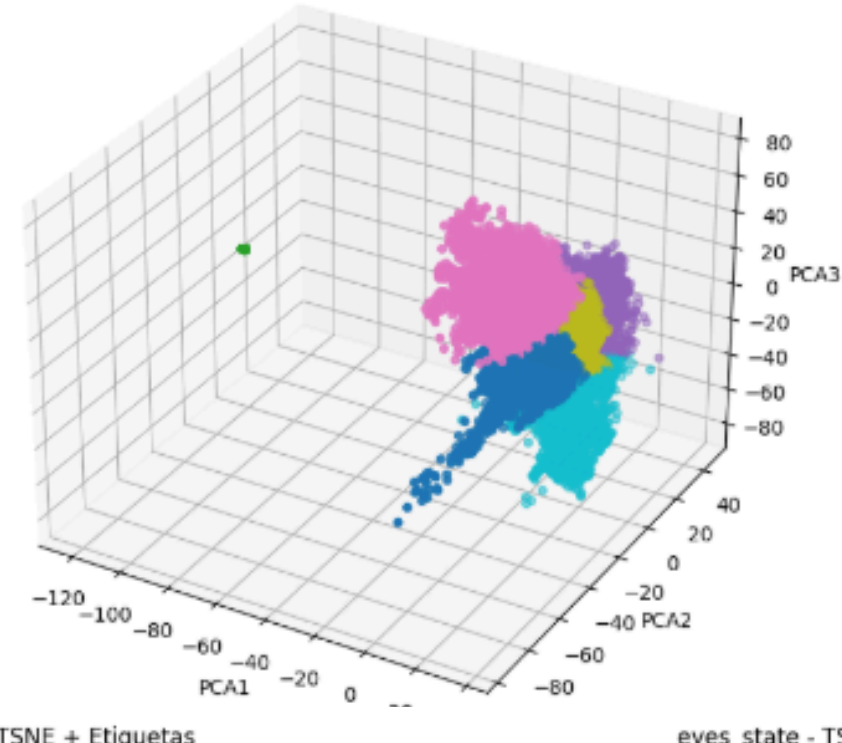
eyes_state - Original Features



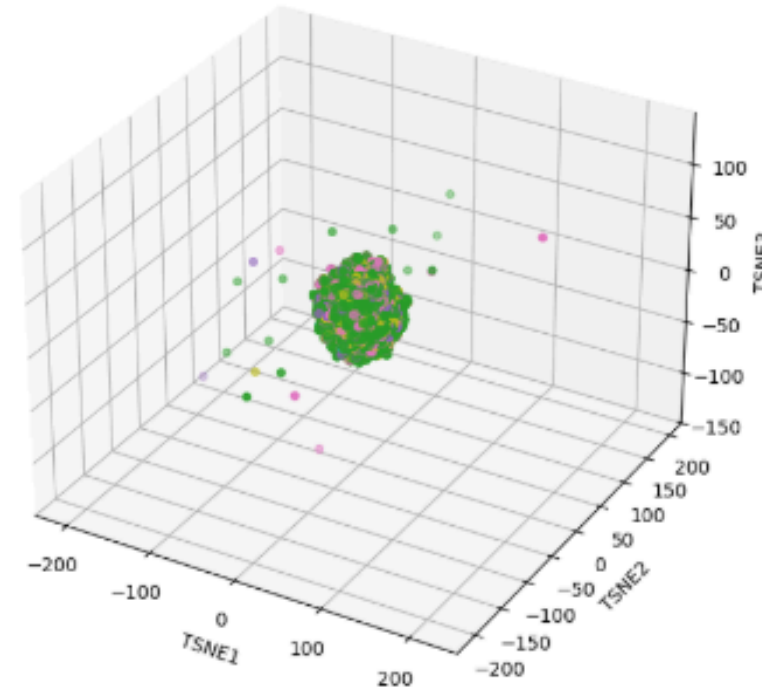
eyes_state - PCA + Etiquetas



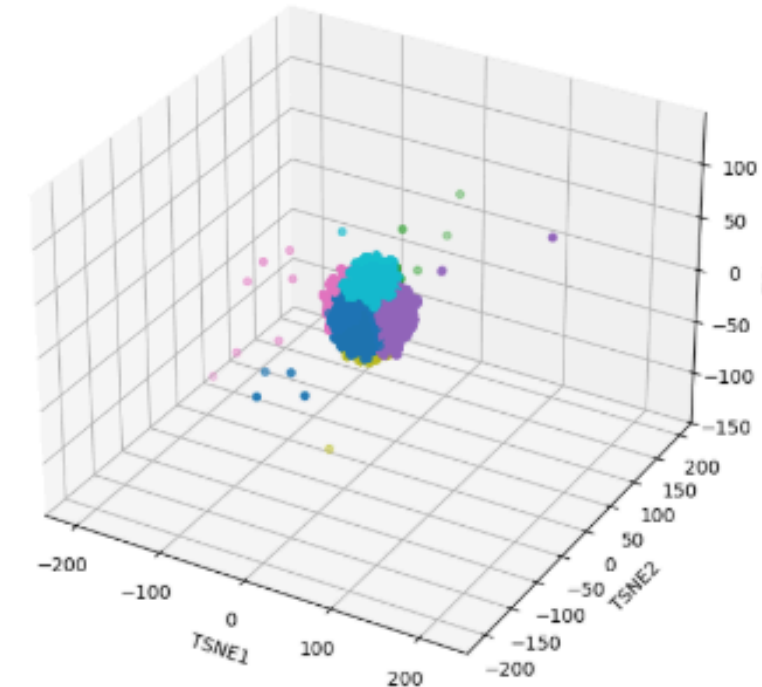
eyes_state - PCA + KMeans



eyes_state - TSNE + Etiquetas

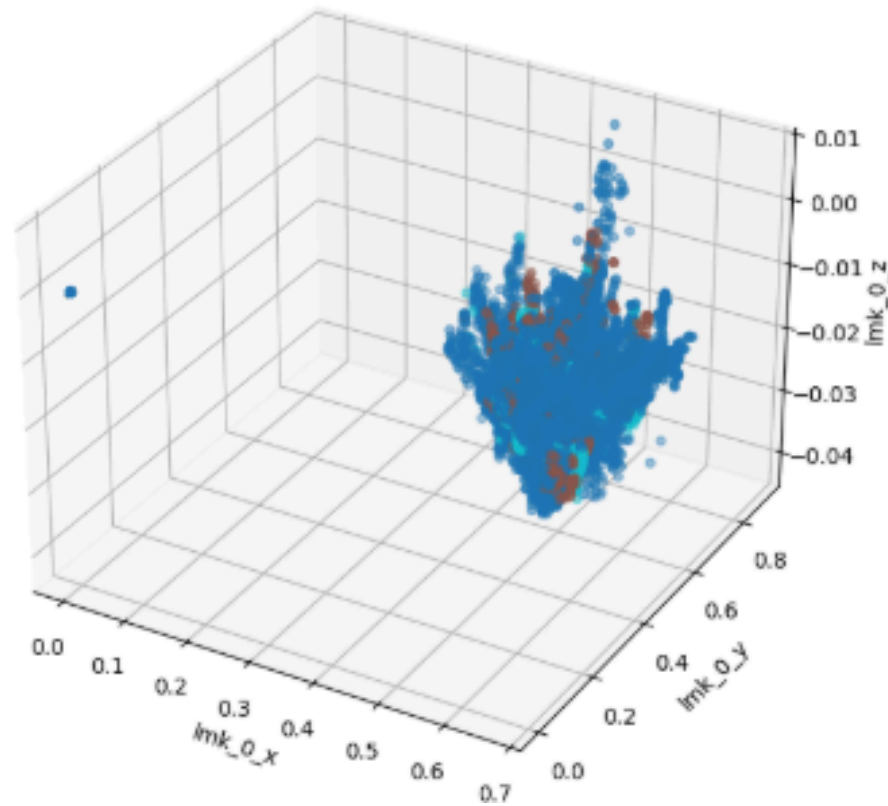


eyes_state - TSNE + KMeans

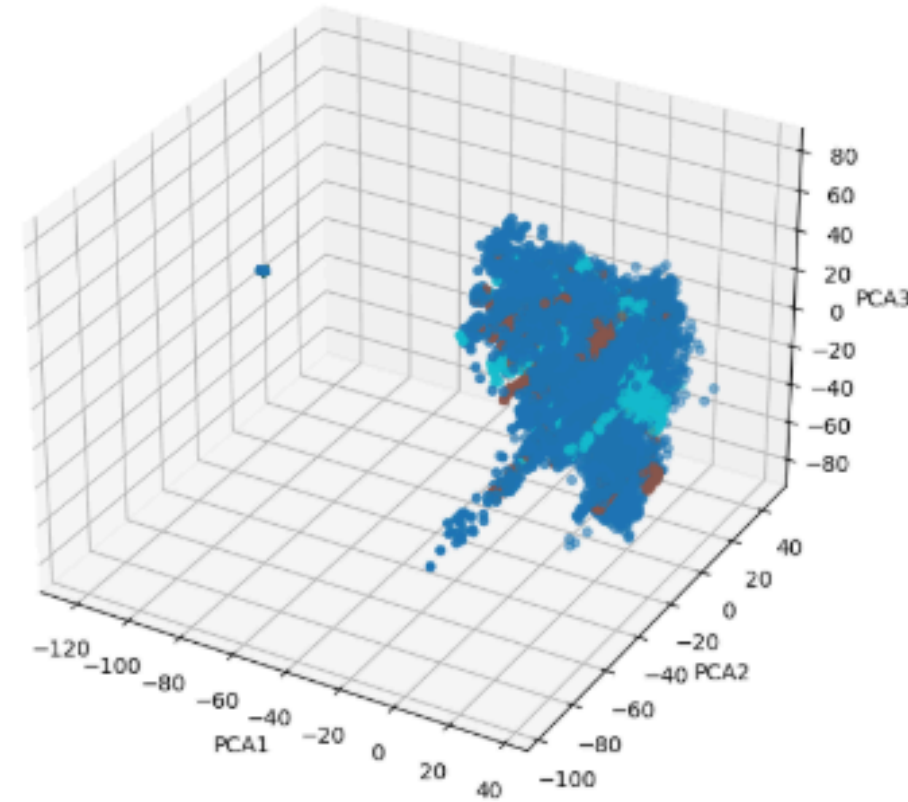


5.3 - Bonus: K-Means en 3D

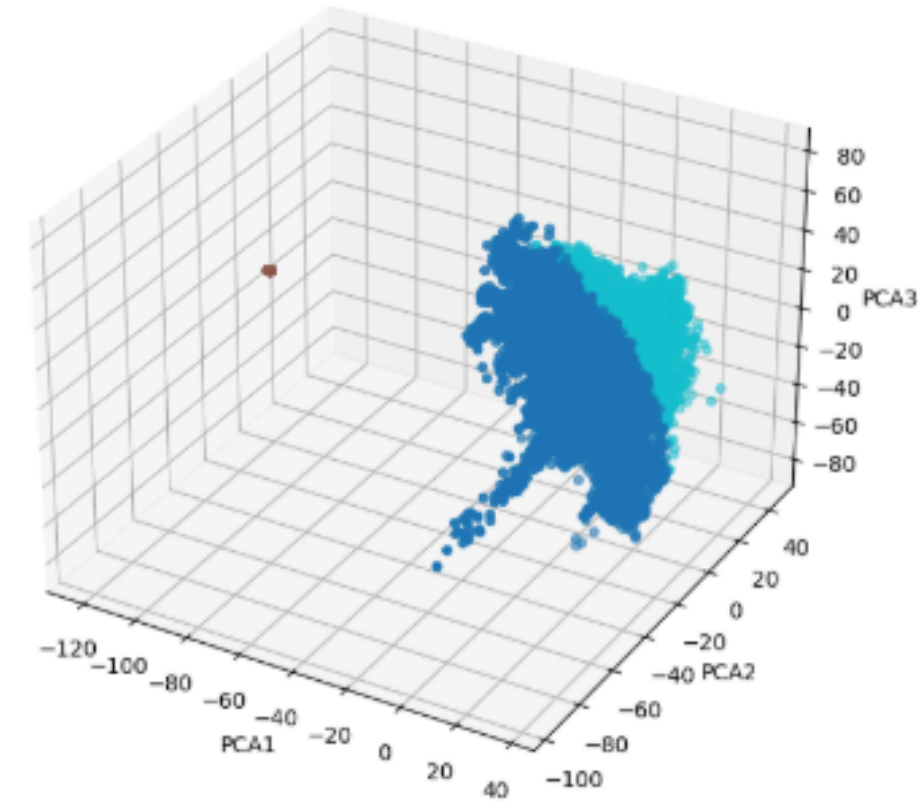
yawning - Original Features



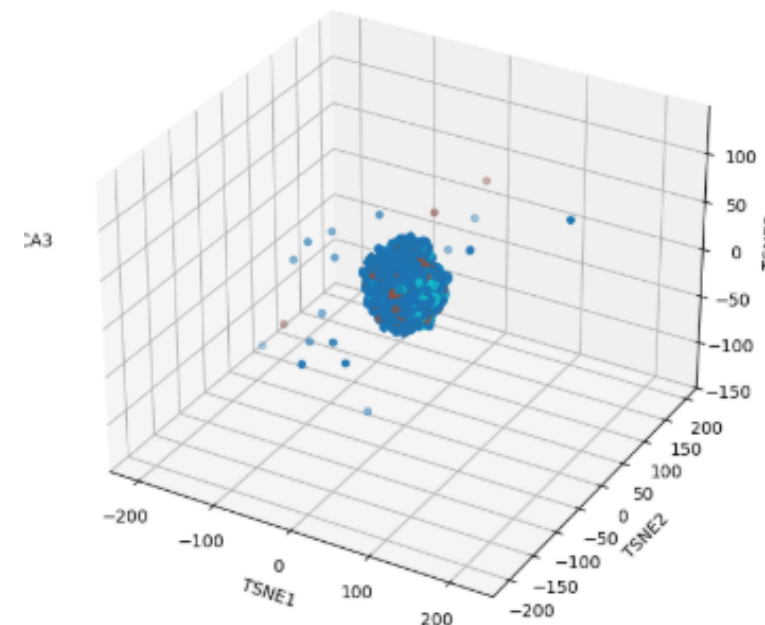
yawning - PCA + Etiquetas



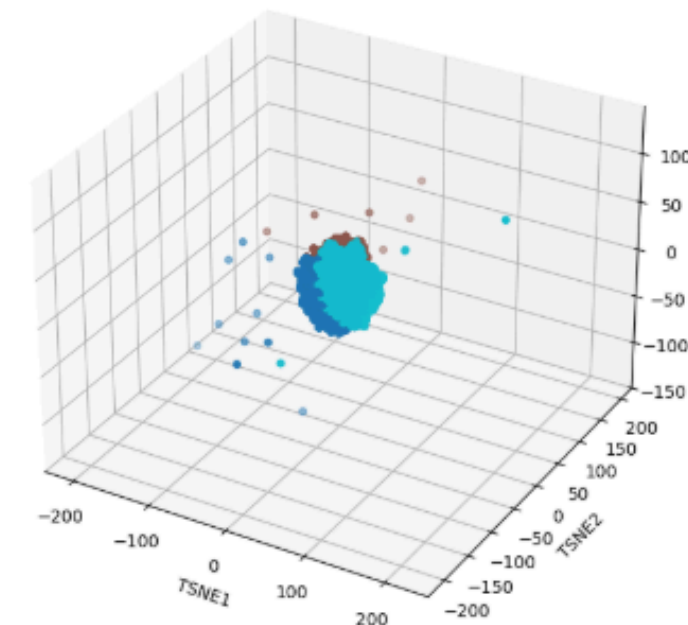
yawning - PCA + KMeans



yawning - TSNE + Etiquetas



yawning - TSNE + KMeans



Conclusiones

- Se observó la necesidad y la gran utilidad de utilizar métodos para la **reducción de dimensionalidad**, mejorando ciertas métricas.

- Sin embargo, la manera agrupada en la que se disponen nuestros datos hace que sean **difíciles de separar para muchos algoritmos de clustering**.



Conclusiones

- Probablemente, tras aplicar alguna transformación a los datos diferentes a las empleadas, sea posible **incrementar la distancia entre las proyecciones**, y mejorar la separación

- Aún así, el **PCA deja muy buenas impresiones**, siendo un algoritmo clásico pero muy útil.



Tablero y Repositorio

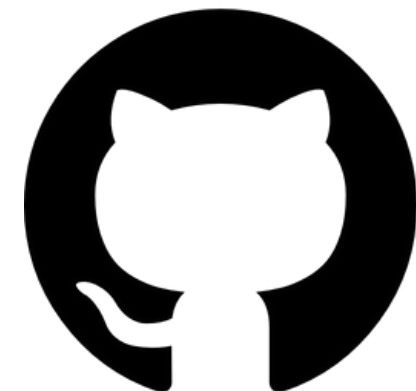
Tablero / Implementación



- <https://colab.research.google.com/drive/1k8VJtLAKs58UR6CDE8fYobNOjEDjQgGS?usp=sharing>

Repositorio

- https://github.com/popcorner893/Sistema_Monitoreo_Conductor





¡Gracias!